

DATASTRUCTURESAND ALGORITHMS

23CSE203

Submitted by	
Name	A. Chethan venkata deva
Roll No	AV.SC.U4CSE25008
Year/sem/section	2nd year/sem-3/CSE-A
Credits	5

Submitted to	
Name	DR.RAJKUMAR BATCHU
Department	School of computing
Designation	Asst. professor(SI. Grade)

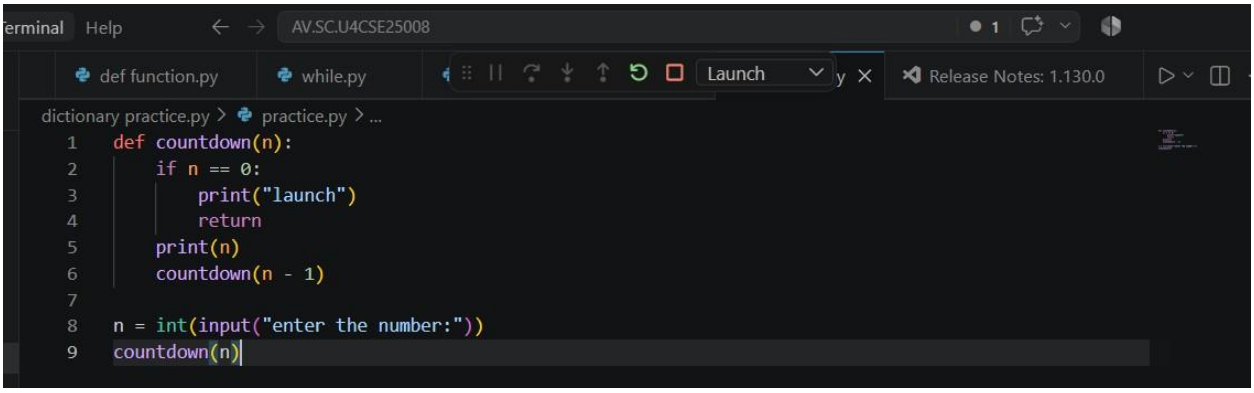
INDEX

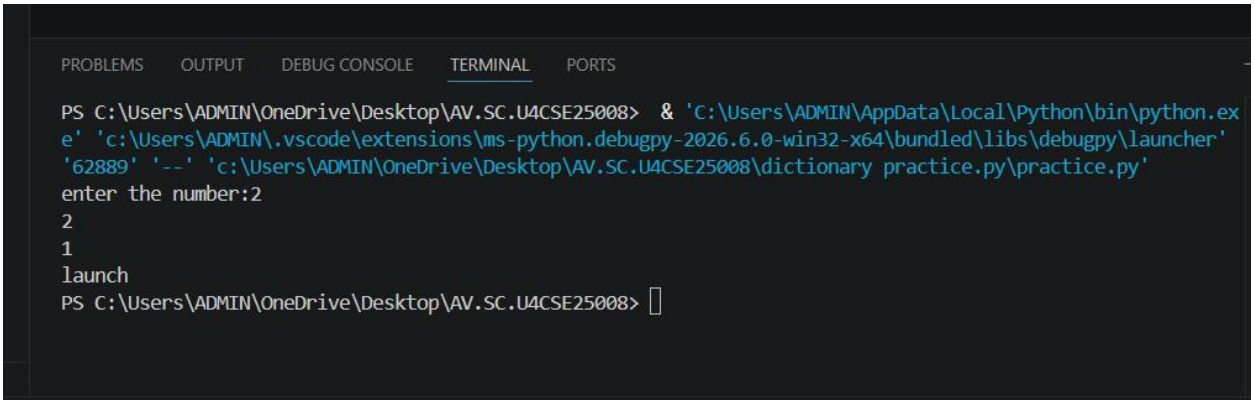
s.no	title	Date
1	WEEK 1: 1a. Recursive countdown from n to Launch. 1b. Recursive calculation of compound growth factor. 1c. Recursive search for employee ID in list. 1d. Recursive factorial calculation for parcel arrangements. 1e. Recursive Fibonacci series generation.	23-07-26
2	WEEK 2: 2a. Write a python program to perform linear search on the give input. 2b. Write a python program to perform binary search on the given. Verify by taking sorted array as an input. 2c. Write a python program to perform binary search on the given. Verify by taking unsorted array as an input.	30-07-26
3	WEEK-3: 3a.write a python program to perform bubblesort on the given input 3b. write a python program to perform insertion_sort on the given input 3c. write a python program to perform selection sort by taking given input 3d.write a python program to perform merge sort by taking the given input . 3e. write a python program to perform quick sort by taking given input	06-08-26
4	Week-4:4a.write a python program to perform linked list operations by taking the input from the user 4b. write a python program to perform double linked list operations by taking input from the user 4c. write a python program to perform circular linked list operations by taking input from the user	13-08-26
5	Week-5: 5a write a python program to implement the stack by using arrays. 5b. write a python program to implement stack using the linked list	27-08-26
6	Week-6: 6a write a python program to implement the queue by suing the arrays. 6b, write a python program to implement the queue by using the linked list.6c. write a python progam to implement the circular queue using the arrays	3-09-26
7	Week-7 :7a write a python program to implement the binary tree using arrays 7b: write a python program to implement the binary tree using linked list	24-09-26
8	Week-8 8a write a python program to implement the binary search tree using the arrays	8-10-26
9	Week-9 write a python program to implement the AVL tree using arrays	22-10-26
10	Week-10 write a python program on hashing separate chaining , linear , quadratic probing	29-10-26

WEEK-1

1A.AIM:A Rocket launch system displays a countdown before launch. Instead of suing a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1 followed by “launch”.

Code:





Explanation

Condition 1: Positive Input (n > 0)

- The function checks whether n == 0.
- Since n is positive, the condition is **False**.
- It executes:
    - print(n) → prints the current value of n.
    - countdown(n-1) → calls the same function again with n-1.
- This process continues until n becomes 0.
- When n == 0, the condition becomes **True**.
- It prints **launch**.
- The return statement stops that function call.
- Then the recursive calls return one by one.
- Since there is no value returned by the function, it returns **None**.

Condition 2: Zero Input (n == 0)

- The function checks if n == 0.
- The condition is immediately **True**.
- It prints **launch**.
- The return statement stops the function.
- No recursive call takes place.
- The function returns **None**.

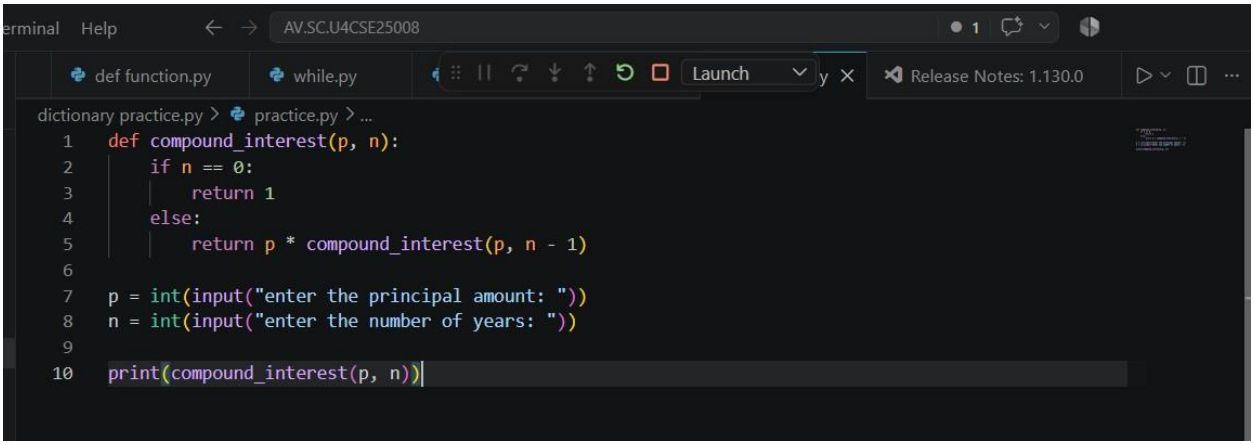
Condition 3: Negative Input (n < 0)

- The condition n == 0 is **False**.
- It prints the negative value.
- Then it calls itself with n-1, making the value even more negative.
- Therefore, it will **never reach 0**.
- The recursion continues indefinitely.
- Eventually, Python raises a **RecursionError** because the maximum recursion depth is exceeded

s.no	Error obtained	Rectification
1	Negative input in countdown	Add a condition to handle negative numbers before recursion.

1B. Aim: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p(n) where p=principal growth, n= no. of years.

Code:





Explanation

Condition 1: n > 0

- The function compound_interest(p, n) is called with the principal amount p and number of years n.
- The function checks whether n == 0.
- Since n is greater than 0, the condition is **False**.
- It executes the else block.
- It calculates:
    - p * compound_interest(p, n-1)
- The function calls itself recursively with n-1.
- This process continues until n becomes 0.
- Each recursive call multiplies p one more time.
- Therefore, the function calculates $\mathbf{p^n}$.

Condition 2: n == 0

- When n becomes 0, the condition n == 0 becomes **True**.
- The function returns 1.
- This 1 acts as the **base case** for the multiplication.
- The recursive calls then return one by one.
- Each returned value is multiplied by p.
- Finally, the result is printed using print(compound_interest(p,n))

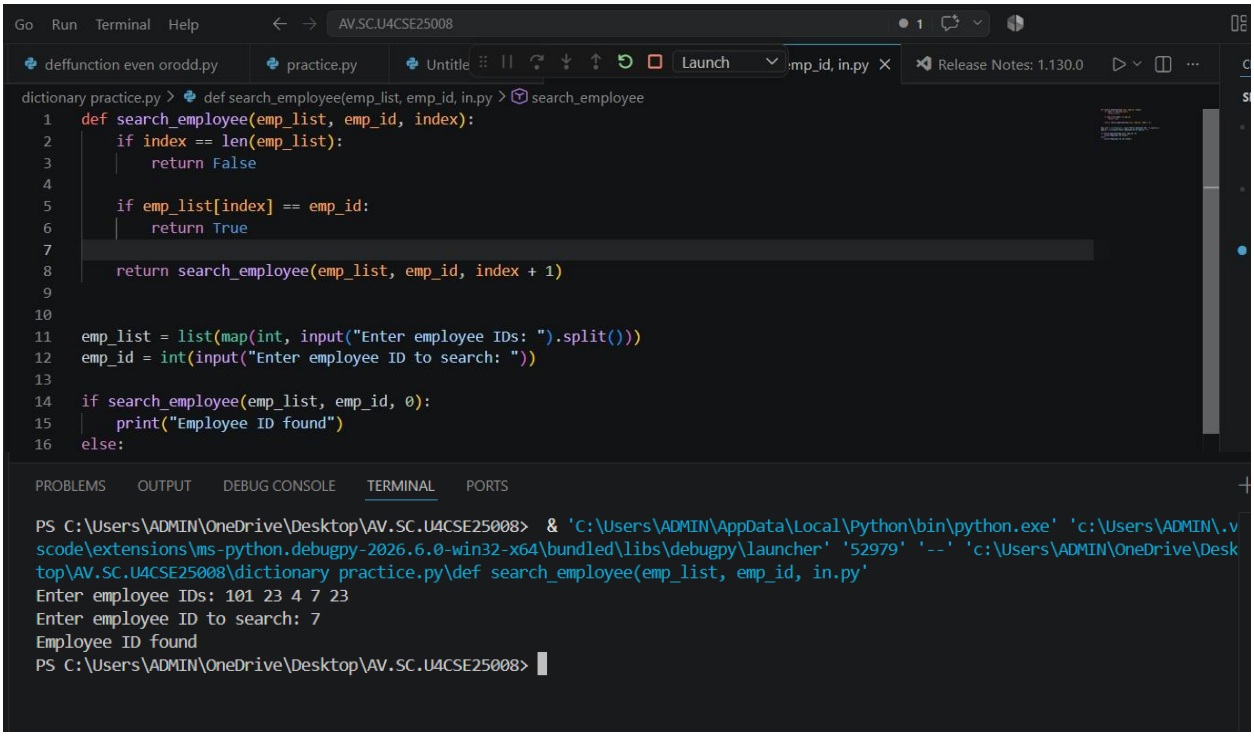
Error table:

s.no	Error obtained	rectification

1	Wrong indentation	Maintain proper indentation.

1C. AIM . An Hr department stores employee IDs in a list .Develop a recursive program to search for a given employee ID write a recursive function to search for an employee ID in a list.

Code:



Condition 1: index == len(emp_list)

- The function checks whether index has reached the length of the employee list.
- If index == len(emp_list), it means we have checked **all employee IDs**.
- The employee ID was not found.
- The function returns **False**.

- No further recursion takes place.

Condition 2: emp_list[index] == emp_id

- The function checks whether the current employee ID is equal to the ID we are searching for.
- If both IDs are equal, the condition becomes **True**.
- The function returns **True**.
- The recursion stops because the employee ID has been found.

Condition 3: Employee ID does not match

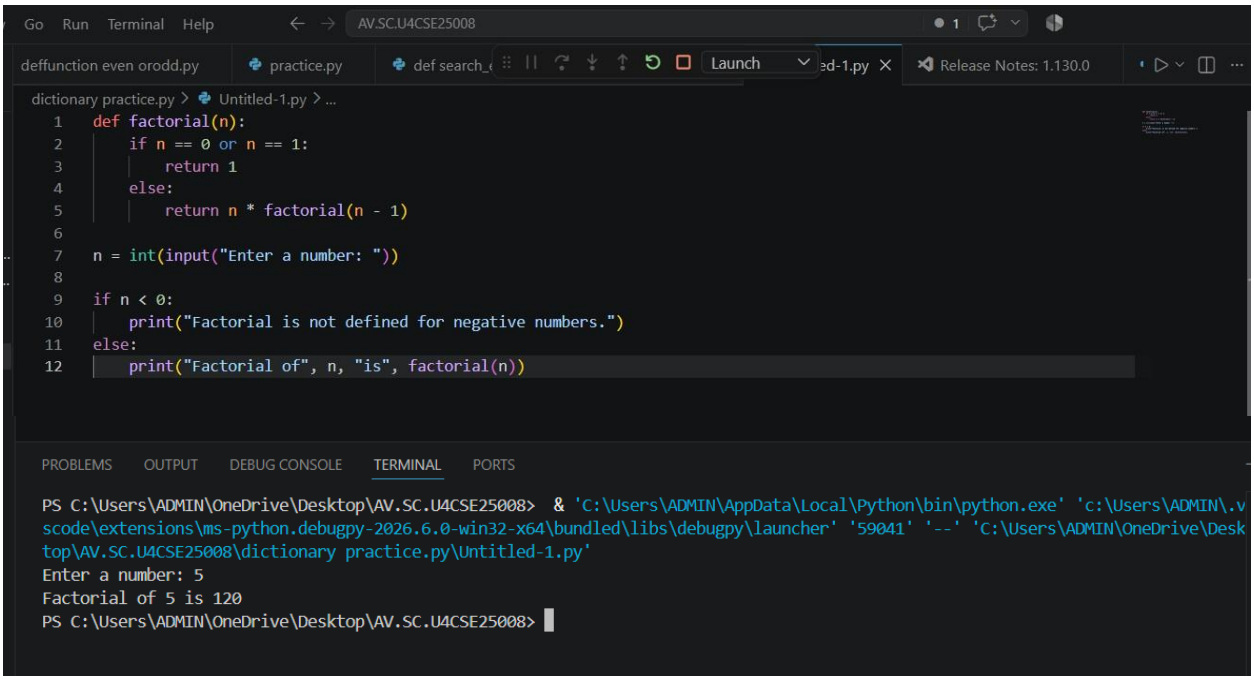
- If the current employee ID is not equal to emp_id, the condition is **False**.
- The function calls itself recursively with:
    - index + 1
- This moves to the **next employee ID** in the list.
- The process continues until:
    - The employee ID is found, or
    - The entire list is checked.

Error table:

s.no	Error obtained	Rectification
1	Missing return statement	Add the required return statement

1D. AIM. In a logistics company , the no . of possible ways to arrange in different parcels is calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

Code:



Condition 1: Negative Input (n < 0)

- The program first checks whether n < 0.
- If n is negative, the condition is **True**.
- Factorial is not defined for negative numbers.
- It prints:
  Factorial is not defined for negative numbers.
- The factorial() function is **not called**.

Condition 2: Base Case (n == 0 or n == 1)

- Inside the factorial() function, it checks:
  n == 0 or n == 1
- If n is 0 or 1, the condition is **True**.
- The function returns **1**.
- This is the **base case** of the recursion.
- It stops further recursive calls.

Condition 3: n > 1

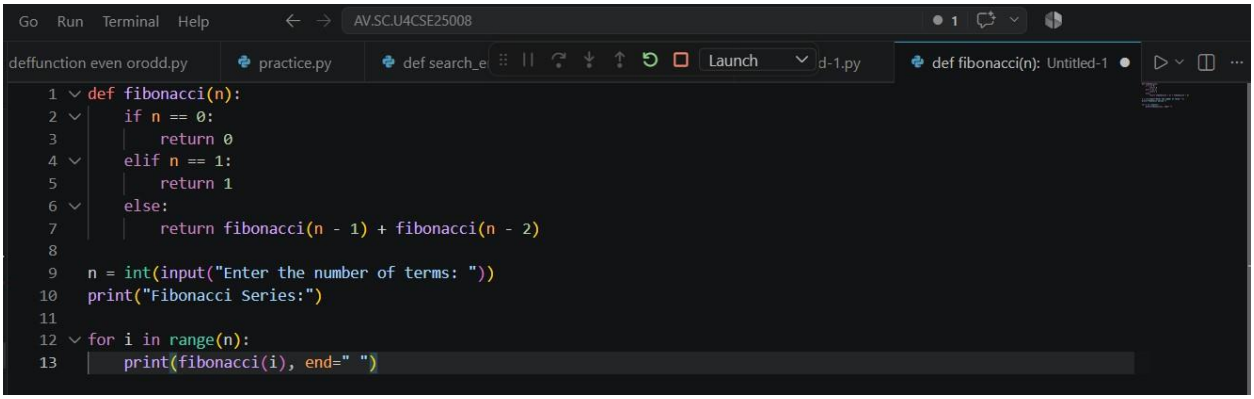
- If n is greater than 1, the condition n == 0 or n == 1 is **False**.
- The else block is executed.
- It returns:
  n * factorial(n - 1)
- The function calls itself with n - 1.
- This continues until n becomes 1.
- Then the returned values are multiplied together

Error table:

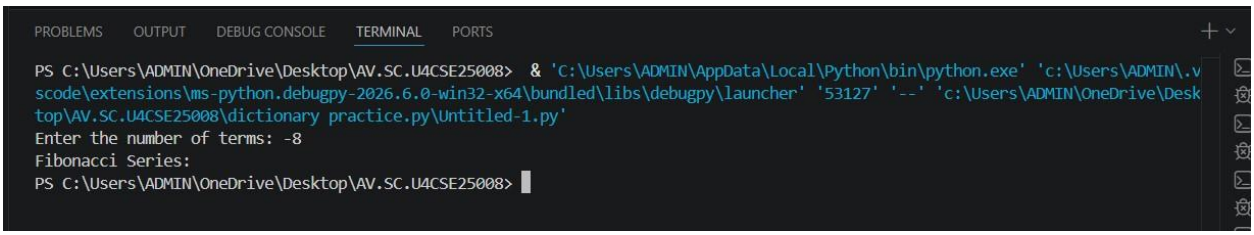
s.no	Error obtained	Rectification
1	Wrong recursive value	Updated the recursive value correctly

1E. AIM. Write a recursive program to print the first n terms of the fibonacci series

Code:







Explanation

Condition 1: n == 0

- The function checks whether n == 0.
- If the condition is **True**, the function returns 0.
- This is the **first base case**.
- No further recursive call takes place.

Condition 2: n == 1

- If n is not 0, the function checks whether n == 1.
- If the condition is **True**, the function returns 1.
- This is the **second base case**.
- No further recursive call takes place

Condition 3: n > 1

- If n is neither 0 nor 1, both conditions are **False**.
- The else block is executed.
- The function calls itself **two times**:
    - fibonacci(n - 1)
    - fibonacci(n - 2)
- The two returned values are added together.

- This process continues until the base cases n == 0 and n == 1 are reached.

Error table:

s.no	Error obtained	Rectification
1	Incorrect Fibonacci recursion	Use fibonacci(n-1) + fibonacci(n-2).

Week-2

2a. AIM: Write a python program to perform linear search on the give input.

Code:

```python
def linear_search(arr, k):
    for i in range(len(arr)):
        if arr[i] == k:
            return i
    return -1

n = int(input("enter the no of elements"))
arr = []

for i in range(n):
    arr.append(int(input("enter the element:")))

k = int(input("enter the search element"))
print(linear_search(arr, k))
```

```
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>  & 'C:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\.v
scode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '56540' '--' 'c:\Users\ADMIN\OneDrive\Desk
top\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
enter the no of elements 2
enter the element:3
enter the element:4
enter the search element 1
-1
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: Element is Found (arr[i] == k)

- The function linear_search(arr, k) receives the array arr and the search element k.
- The for loop starts from the first element of the array.
- It checks each element one by one using:
  arr[i] == k
- If the current element is equal to k, the condition becomes **True**.
- The function immediately returns the **index i**.
- Since the function returns the index, the loop stops immediately.
- The remaining elements are not checked.

Condition 2: Element is Not Found

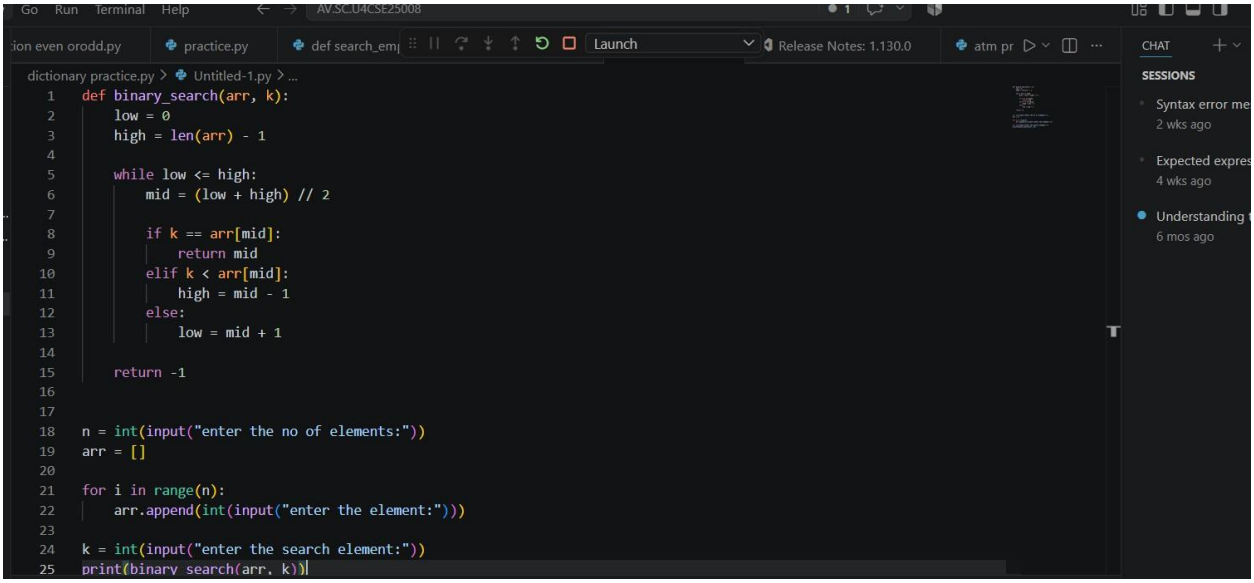
- The loop checks every element in the array.
- If none of the elements is equal to k, the if condition is always **False**.
- After the loop finishes, the statement return -1 is executed.
- -1 indicates that the search element is **not present** in the array.

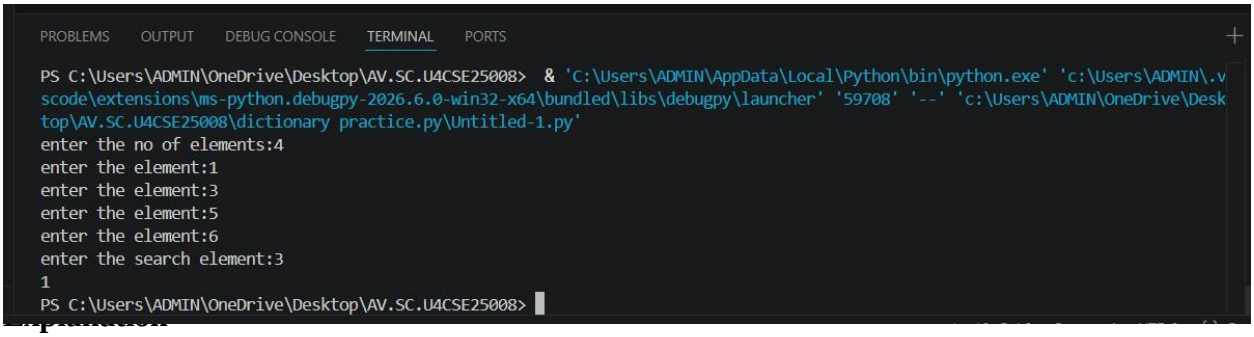
Error table:

s.no	Error obtained	Rectification
1	Wrong loop range	Use range(n) to process exactly n elements.

2b. AIM: Write a python program to perform binary search on the given. Verify by taking sorted array as an input.

Code:





Explanation

Condition 1: k == arr[mid] — Element Found

- The function binary_search(arr, k) receives the array and the search element.

- low is initialized to 0, which represents the first index.
- high is initialized to len(arr)-1, which represents the last index.
- The while loop runs as long as low <= high.
- mid is calculated using:
  mid = (low + high) // 2
- The middle element is compared with the search element k.
- If k == arr[mid], the element is found.
- The function returns the **index mid** and stops searching.

Condition 2: `k < arr[mid]` — Search Left Half

- If `k` is smaller than the middle element, the condition is **True**.
- This means the search element can only be present in the **left half**.
- Therefore:
  `high = mid - 1`
- The right half is eliminated.
- The process continues with the remaining left half

Condition 3: k > arr[mid] — Search Right Half

- If k is greater than the middle element, the elif condition is **False**.
- The else block is executed.
- This means the search element can only be present in the **right half**.
- Therefore:
  low = mid + 1
- The left half is eliminated.
- The process continues with the remaining right half.

Condition 4: low > high — Element Not Found

- The while loop continues until low <= high becomes **False**.
- When low > high, there are no elements left to search.
- This means the search element is not present in the array.
- The function returns **-1**.

Error table:

s.no	Error obtained	Rectification

```python
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1

    return -1


arr = list(map(int, input("Enter the elements: ").split()))

if arr == sorted(arr):
    print("Array is already sorted.")
else:
    print("Array is unsorted.")
    arr.sort()
    print("Array after sorting:", arr)

key = int(input("Enter the element to search: "))

result = binary_search(arr, key)

if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found.")
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> ^C
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>  c:; cd 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008'; & 'C:\Users\ADMIN\AppData\Local\Python\bin\python.exe' 'c:\Users\ADMIN\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55895' '--' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\Untitled-1.py'
Enter the elements: 1 4 2 6 8
Array is unsorted.
Array after sorting: [1, 2, 4, 6, 8]
Enter the element to search: 2
Element found at index: 1
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: arr[mid] == key — Element Found

- The function first calculates the middle index using mid = (low + high) // 2.
- It compares the middle element arr[mid] with the search element key.

- If arr[mid] == key, the condition is **True**.
- The element is found.
- The function returns the index mid.
- The search stops.

Condition 2: key < arr[mid] — Search Left Half

- If key is smaller than arr[mid], the condition is **True**.
- This means the required element may be present in the **left half**.
- The right half is ignored.
- high is changed to mid - 1.
- The search continues in the left half.

Condition 3: key > arr[mid] — Search Right Half

- If key is greater than arr[mid], the previous conditions are **False**.
- The element may be present in the **right half**.
- The left half is ignored.
- low is changed to mid + 1.
- The search continues in the right half.

Condition 4: low > high — Element Not Found

- The while low <= high loop continues until there are elements left to search.
- When low > high, the loop stops.
- This means the element is not present in the array.
- The function returns -1.
- The program prints **"Element not found."**

Condition 5: Array is Already Sorted

- The program checks whether the given array is already sorted.
- It uses:
  arr == sorted(arr)
- If the condition is **True**, it prints:
  "Array is already sorted."

- The array is directly used for binary search.

Condition 6: Array is Unsorted

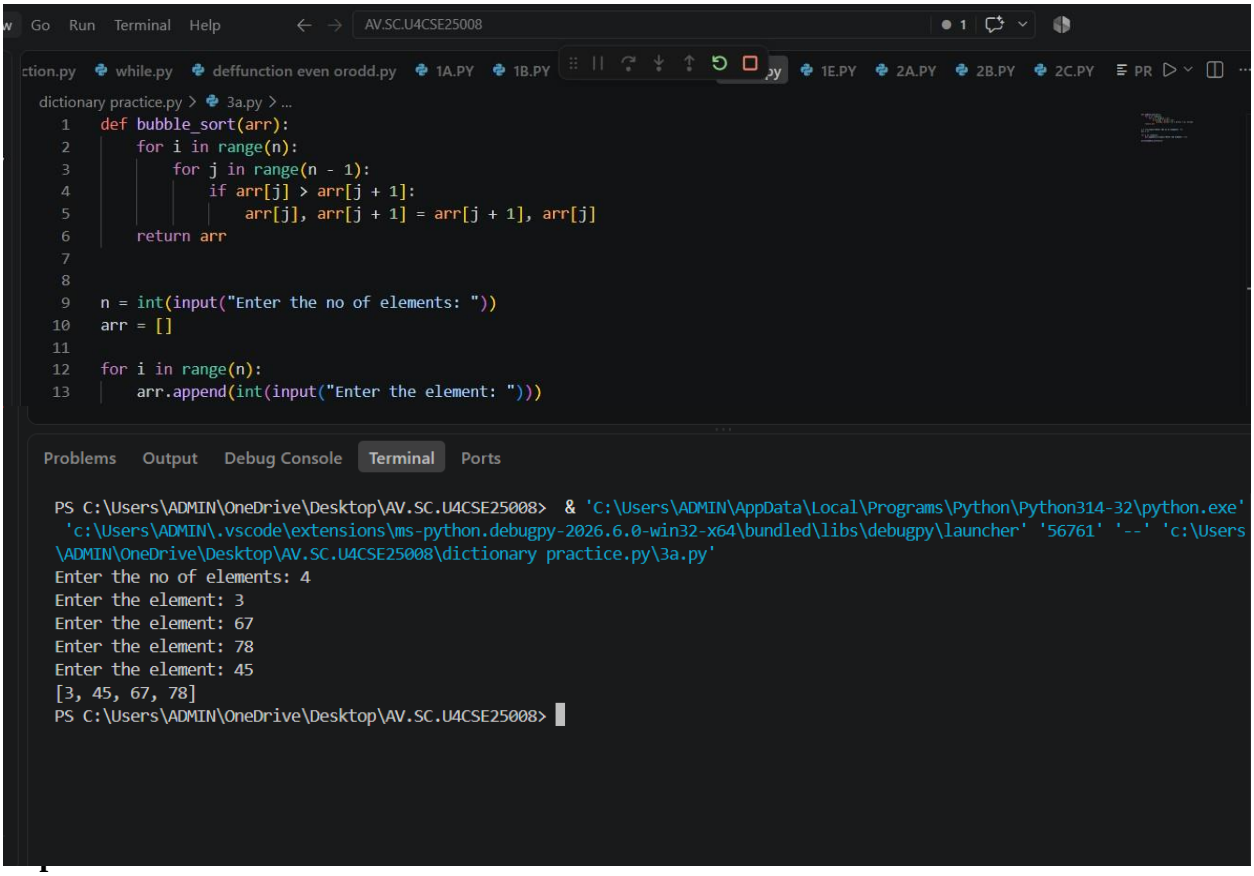
- If arr == sorted(arr) is **False**, the array is unsorted.
- The program prints:
  "Array is unsorted."
- It then uses arr.sort() to sort the array.
- The sorted array is displayed.
- Binary search is then performed on the sorted array.

Error table:

s.no	Error obtained	Rectification
1	Binary search on unsorted array	sort the array first using arr.sort() or provide a sorted array.

Week-3

3a.Aim.Write a python program to sort the elements using a bubble sort technique.
Take an input elements from the user. Code:



Condition 1: arr[j] > arr[j+1] — Elements Need to Be Swapped

- The function bubble_sort(arr) receives the array as an argument.
- The outer for loop runs from 0 to n-1 and controls the number of passes.
- The inner for loop runs from 0 to n-2 and compares adjacent elements.
- The condition arr[j] > arr[j+1] checks whether the current element is greater than the next element.
- If the condition is **True**, the two elements are in the wrong order.
- Therefore, the two elements are swapped using:

- Arr[j],arr[j+1]=arr[j+1],arr[j]
- After the swap, the smaller element moves toward the left and the larger element moves toward the right.

Condition 2: arr[j] <= arr[j+1] — No Swap Required

- If arr[j] > arr[j+1] is **False**, it means the current element is smaller than or equal to the next element.
- Therefore, the elements are already in the correct ascending order.
- No swapping is performed.
- The inner loop continues to compare the next pair of adjacent elements.

Condition 3: for i in range(n) — Repeated Passes

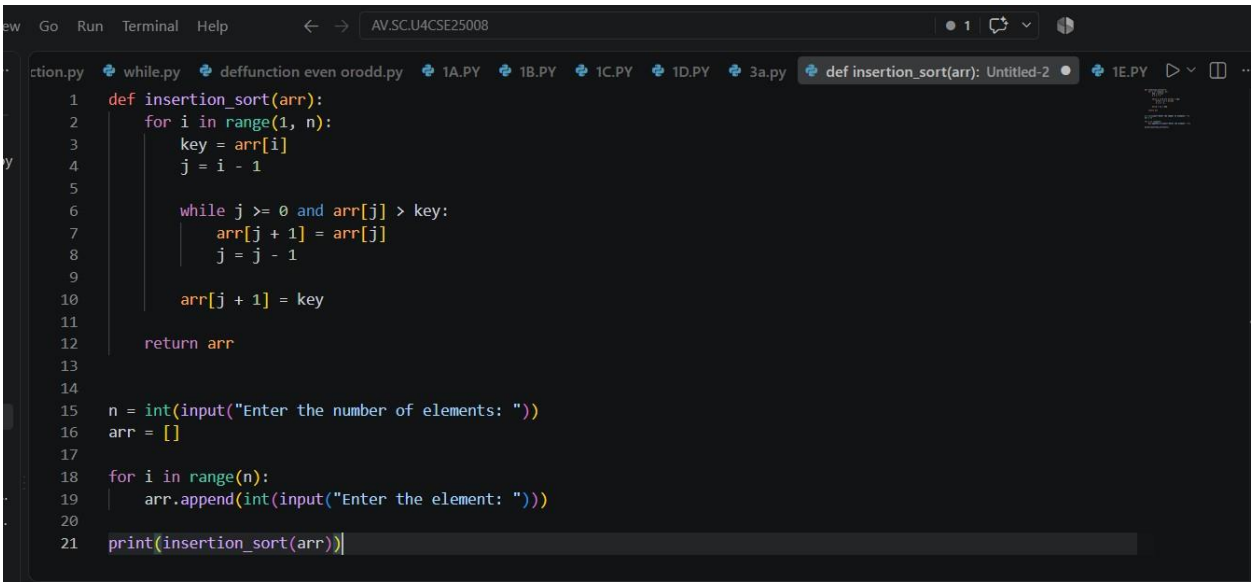
- The outer loop repeats the comparison process for all the elements.
- After each complete pass, the **largest unsorted element moves to the end of the array**.
- The process is repeated until all elements are arranged in ascending order.
- Finally, the function returns the sorted array using:

Error table:

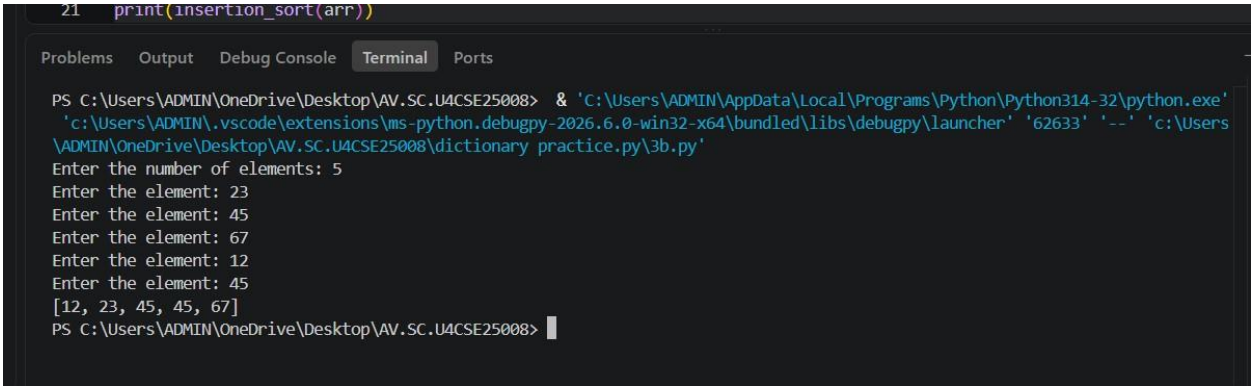
sno	Error obtained	Rectification
1	IndexError: list index out of range	Use correct loop range

3b.Aim. write a python program to sort the elements using a insertion sort technique. Take an input elements from the user.

Code:



Output:



Explanation:

Condition 1: for i in range(1, len(arr)) — Selecting Element

- The loop starts from the **second element**.
- The first element is considered already sorted.
- i represents the element that needs to be placed correctly.

Condition 2: key = arr[i] — Storing Element

- The current element is stored in key.

key = arr[i]

- This element will be compared with the elements before it.

Condition 3: j = i - 1 — Getting Previous Position

- j points to the element just before the current element.

j = i - 1

Condition 4: while j >= 0 and key < arr[j] — Comparing Elements

- The current element is compared with the previous elements.
- If the previous element is greater than key, it is moved one position to the right.

```
while j >= 0 and key < arr[j]:
    arr[j + 1] = arr[j]
    j -= 1
```

- This creates space for the current element.

Condition 5: arr[j + 1] = key — Inserting Element

- The key is placed in its correct position.

arr[j + 1] = key

- The elements before the current position are now sorted.

Condition 6: return arr — Returning Sorted Array

- After all elements are processed, the function returns the sorted array.

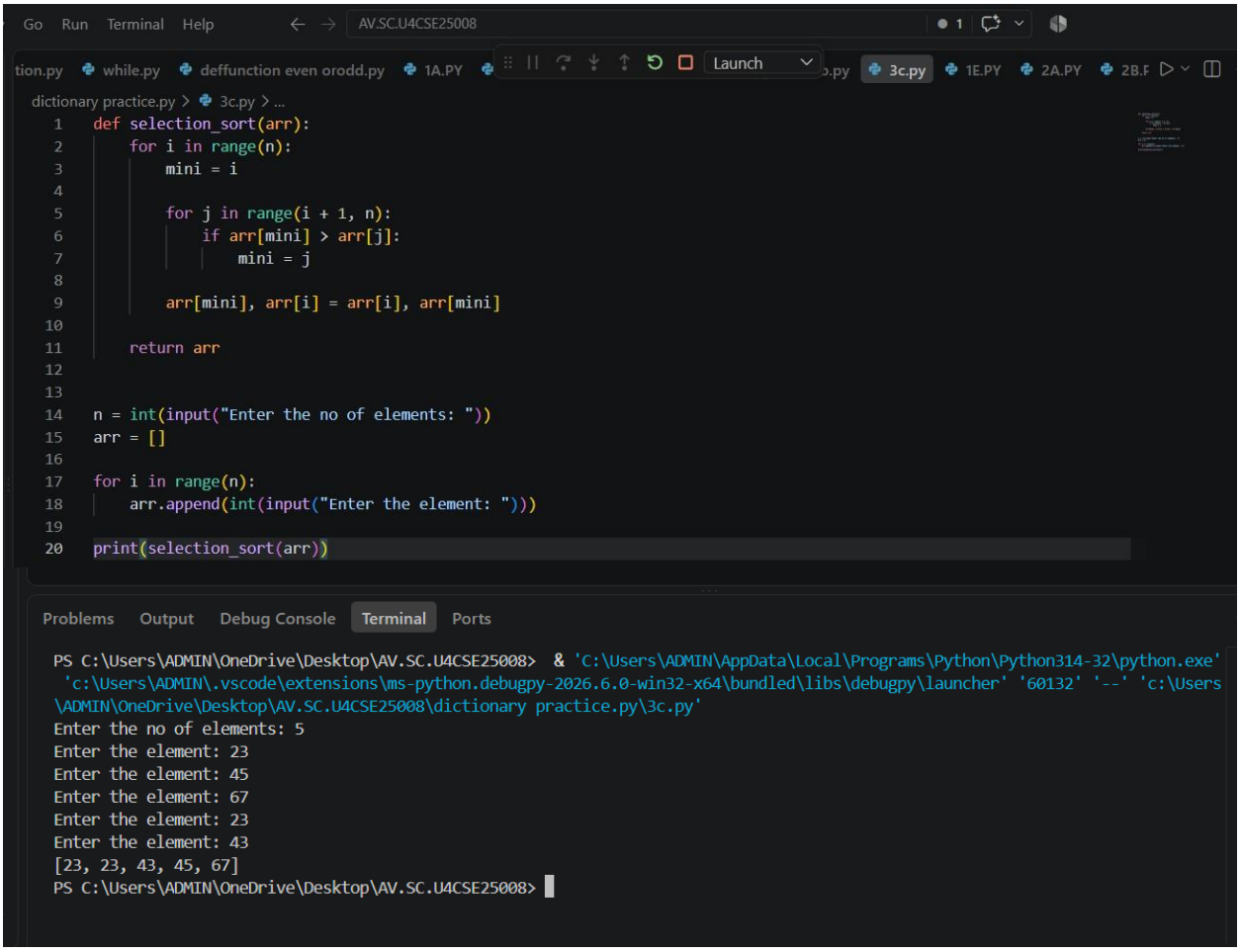
return arr

error table:

sno	Error obtained	Rectification
1	Elements not sorted correctly	Shift elements before insertion

3c.write a python program to sort the elements using selection sort technique . take an input from the user.

Code:



Explanation:

Condition 1: for i in range(n) — Selecting Position

- The loop goes through each position in the array.
- i represents the current position where the smallest element will be placed.

Condition 2: mini = i — Assuming Minimum

- Initially, the element at index i is assumed to be the smallest.

Condition 3: for j in range(i+1,n) — Finding Minimum

- The loop checks all elements after position i.
- It searches for the smallest element.

Condition 4: if arr[mini] > arr[j] — Comparing

- Compares the current minimum with arr[j].
- If arr[j] is smaller, mini is updated.

Condition 5: mini = j — Updating Minimum

- Stores the index of the newly found smallest element.

Condition 6: arr[mini], arr[i] = arr[i], arr[mini] — Swapping

- The smallest element is swapped with the element at position i.
- This places the smallest element in its correct position.

Condition 7: return arr — Returning Result

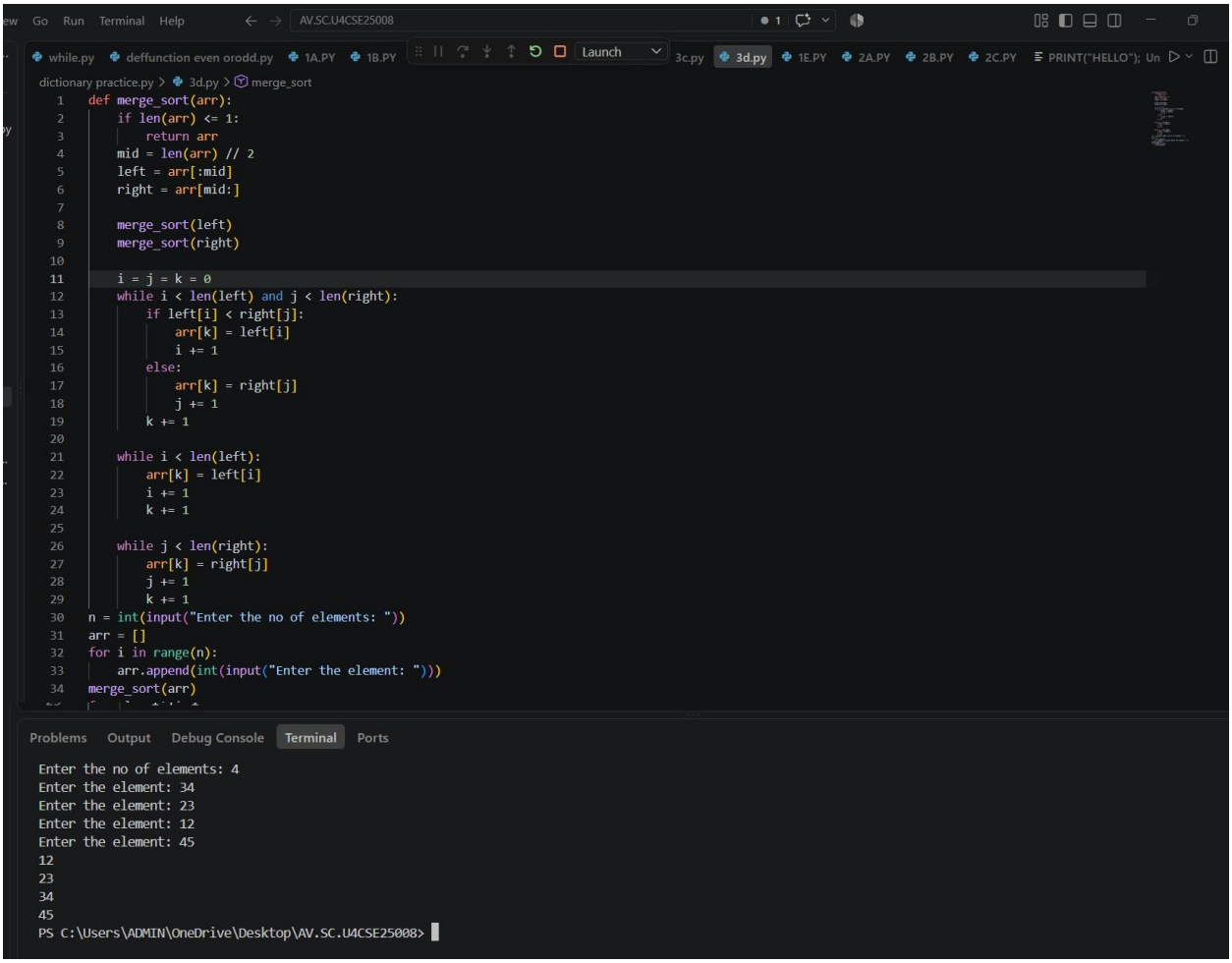
- Returns the completely sorted array.

Error table:

Sno	Error obtained	Rectification
1	Wrong minimum value	Update min_index properly

3d.write a python program to sort the elements using the merge sort technique . take an input from the user.

Code:



Explanation:

Condition 1: if len(arr) <= 1 — Checking the Base Condition

- The function first checks whether the array contains **one or zero elements**.

```
if len(arr) <= 1:
```

  return arr

- If the array has one or zero elements, it is already sorted.
- No further division or comparison is required.
- The function returns the array as it is:

```
return arr
```

Condition 2: mid = len(arr) // 2 — Finding the Middle

- The array is divided into two parts.
- The middle position is calculated using:

```
mid = len(arr) // 2
```

- len(arr) gives the total number of elements.
- // 2 performs integer division and finds the middle index.

Condition 3: left = arr[:mid] — Creating the Left Part

- The elements before the middle position are stored in left.

```
left = arr[:mid]
```

Condition 4: right = arr[mid:] — Creating the Right Part

- The elements from the middle position to the end are stored in right.

```
right = arr[mid:]
```

Condition 5: merge_sort(left) and merge_sort(right) — Sorting Both Parts

- The left and right parts are recursively sorted.

```
merge_sort(left)
merge_sort(right)
```

- The array continues to be divided until each part contains only **one element**.
- A single element is already sorted.
- The smaller sorted parts are then merged together.

Condition 6: i = j = k = 0 — Initializing Indexes

- Three variables are initialized:

```
i = j = k = 0
```

- i keeps track of the current element in the **left array**.
- j keeps track of the current element in the **right array**.
- k keeps track of the position in the **original array** where the element will be placed.

Condition 7: while i < len(left) and j < len(right) — Comparing Both Parts

- The program compares elements from the left and right arrays.

```
while i < len(left) and j < len(right):
```

- The loop continues as long as both arrays still have elements to compare.
- This allows the program to select the smaller element and place it into the original array.

Condition 8: if left[i] < right[j] — Comparing Elements

- The current elements of the left and right arrays are compared.

```
if left[i] < right[j]:
```

- If the left element is smaller than the right element, it is placed into the original array.

```
arr[k] = left[i]
```

- Then i is increased:

i += 1

- This moves to the next element of the left array.

Condition 9: else — Taking the Right Element

- If the right element is smaller than or equal to the left element, the right element is placed into the original array.

else:

arr[k] = right[j]

- Then j is increased:

j += 1

- This moves to the next element of the right array.

Condition 10: k += 1 — Moving to the Next Position

- After placing an element into the original array, k is increased.

k += 1

Condition 11: while i < len(left) — Copying Remaining Left Elements

- Sometimes the right array becomes empty before the left array.

while i < len(left):

- Any remaining elements in the left array are copied into the original array.

arr[k] = left[i]

- Then both i and k are increased:

i += 1

k += 1

- The remaining elements are already sorted, so no comparison is needed.

Condition 12: while j < len(right) — Copying Remaining Right Elements

- Sometimes the left array becomes empty before the right array.

while j < len(right):

- Any remaining elements in the right array are copied into the original array.

arr[k] = right[j]

- Then j and k are increased:

j += 1

k += 1

- This completes the merging process.

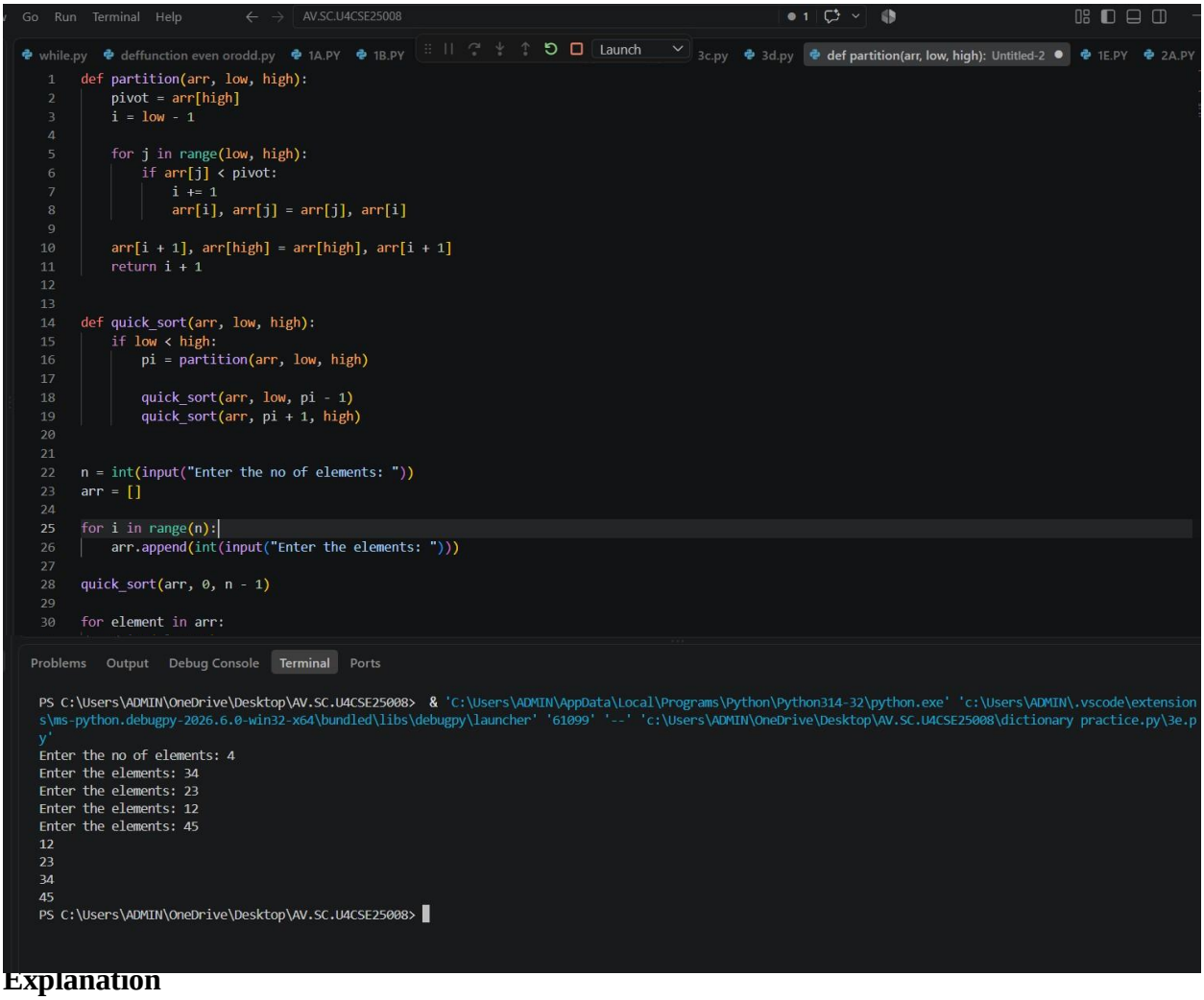
Error table:

sno	Error obtained	Rectification
1	Recursive error	Use if len(arr) <= 1
2	IndexError	Check left/right array indices

Bottom of Form

3e. Aim. Write a python program to sort the elements using a quick sort technique . take an input elements from the user

Code:



```
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1

    for j in range(low, high):
        if arr[j] < pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1


def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)

        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)


n = int(input("Enter the no of elements: "))
arr = []

for i in range(n):
    arr.append(int(input("Enter the elements: ")))

quick_sort(arr, 0, n - 1)

for element in arr:
```

```
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008> & 'C:\Users\ADMIN\AppData\Local\Programs\Python\Python314-32\python.exe' 'c:\Users\ADMIN\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '61099' '--' 'c:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008\dictionary practice.py\3e.py'
Enter the no of elements: 4
Enter the elements: 34
Enter the elements: 23
Enter the elements: 12
Enter the elements: 45
12
23
34
45
PS C:\Users\ADMIN\OneDrive\Desktop\AV.SC.U4CSE25008>
```

Explanation

Condition 1: if arr[j] < pivot — Element Smaller Than Pivot

- The function partition(arr, low, high) is used to divide the array into two parts.
- pivot = arr[high] selects the **last element** of the array as the pivot.
- i = low - 1 is initialized to keep track of the position where elements smaller than the pivot should be placed.
- The for loop checks each element from index low to high-1.
- The condition arr[j] < pivot checks whether the current element is smaller than the pivot.
- If the condition is **True**, i is increased by 1.
- The current element is then swapped with the element at index i:

arr[i], arr[j] = arr[j], arr[i]

- This places all elements smaller than the pivot toward the left side.

Condition 2: arr[j] >= pivot — No Swap Required

- If arr[j] < pivot is **False**, it means the current element is greater than or equal to the pivot.
- No swapping is performed.
- The loop continues to check the next element.
- These elements will remain on the right side of the pivot after partitioning.

Condition 3: arr[i+1], arr[high] = arr[high], arr[i+1] — Placing the Pivot

- After all elements have been checked, the pivot is placed in its correct position.
- The pivot is swapped with the element at index i+1:

arr[i+1], arr[high] = arr[high], arr[i+1]

- After this operation:
  - Elements smaller than the pivot are on the **left**.
  - The pivot is in its **correct position**.
  - Elements greater than or equal to the pivot are on the **right**.
- The function returns the pivot position:

return i+1

Condition 4: if low < high — Continue Quick Sort

- The function quick_sort(arr, low, high) is used to sort the array.
- The condition low < high checks whether the current part of the array contains more than one element.
- If the condition is **True**, the array needs to be sorted.
- The partition() function is called:

pi = partition(arr, low, high)

- pi stores the final position of the pivot.

Condition 5: quick_sort(arr, low, pi-1) — Sort Left Part

- After partitioning, the elements on the left side of the pivot are sorted separately.
- This is done using:

quick_sort(arr, low, pi-1)

- The pivot itself is not included because it is already in its correct position.

Condition 6: quick_sort(arr, pi+1, high) — Sort Right Part

- The elements on the right side of the pivot are sorted separately.
- This is done using:

quick_sort(arr, pi+1, high)

- Again, the pivot is not included because it is already correctly positioned.

Input Section

- n = int(input("enter the no of elements:")) takes the number of elements from the user.
- arr = [] creates an empty list to store the elements.
- The for loop runs n times and takes each element from the user:
- for i in range(n):

arr.append(int(input("enter the elements:")))

- Each entered value is converted into an integer and added to the array using append().

Calling the Quick Sort Function

quick_sort(arr, 0, n-1)

- This calls the quick_sort() function.

- 0 represents the first index of the array.
- n-1 represents the last index of the array.
- The complete array is therefore passed for sorting.

.

Error table:

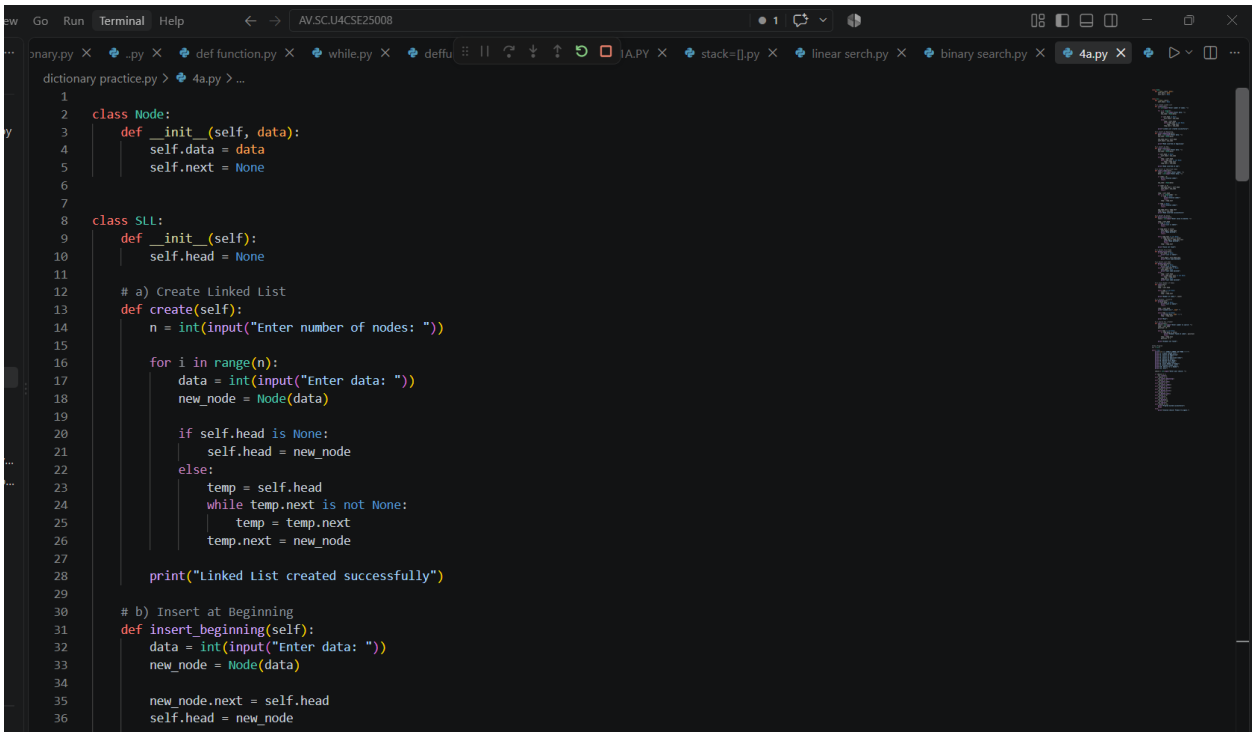
sno	Error obtained	Rectification
1	IndexError	Check i, j, and pivot boundaries
2	Wrong swapping conditions	By shifting to the correct spaces

LAB -4

Aim 4.a:Write a program to implement a single linked list which would perform the below operations, by taking an input from the user.

a)create a linked list
b)insert at beginning
c)insert at end
d)insert at a specified index
e)delete by value
f)delete 1st node
g)delete last node
h)count no.of nodes
i)display or traverse
j)search for a number
k)exit

Program:



```
    print("Node inserted at beginning")

# c) Insert at End
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
    else:
        temp = self.head
        while temp.next is not None:
            temp = temp.next
        temp.next = new_node

    print("Node inserted at end")

# d) Insert at Specified Index
def insert_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter data: "))

    if index < 0:
        print("Invalid index")
        return

    new_node = Node(data)

    if index == 0:
        new_node.next = self.head
        self.head = new_node
        return
    temp = self.head
    for i in range(index - 1):
        if temp is None:
            print("Invalid index")
            return
        temp = temp.next

    if temp is None:
        print("Invalid index")
        return

    new_node.next = temp.next
    temp.next = new_node
    print("Node inserted successfully")

# e) Delete by Value
def delete_value(self):
    value = int(input("Enter value to delete: "))

    temp = self.head
    if temp is None:
        print("List is empty")
        return

    if temp.data == value:
        self.head = temp.next
        print("Node deleted")
        return

    while temp.next is not None:
        if temp.next.data == value:
            temp.next = temp.next.next
            print("Node deleted")
            return
```

```
    while temp.next is not None:
        if temp.next.data == value:
            temp.next = temp.next.next
            print("Node deleted")
            return
        temp = temp.next

    print("Value not found")

# f) Delete First Node
def delete_first(self):
    if self.head is None:
        print("List is empty")
    else:
        self.head = self.head.next
        print("First node deleted")

# g) Delete Last Node
def delete_last(self):
    if self.head is None:
        print("List is empty")
    elif self.head.next is None:
        self.head = None
        print("Last node deleted")
    else:
        temp = self.head
        while temp.next.next is not None:
            temp = temp.next
        temp.next = None
        print("Last node deleted")

# h) Count Number of Nodes
def count(self):
    count = 0
```

```
    # g) Delete Last Node
    def delete_last(self):
        if self.head is None:
            print("List is empty")
        elif self.head.next is None:
            self.head = None
            print("Last node deleted")
        else:
            temp = self.head
            while temp.next.next is not None:
                temp = temp.next
            temp.next = None
            print("Last node deleted")

    # h) Count Number of Nodes
    def count(self):
        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)

    # i) Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head
        print("Linked List:", end=" ")
```

```
    # j) Search for a Number
    def search(self):
        value = int(input("Enter number to search: "))
        temp = self.head
        position = 0

        while temp is not None:
            if temp.data == value:
                print("Element found at index", position)
                return
            temp = temp.next
            position += 1

        print("Element not found")


# Main Program
sll = SLL()

while True:
    print("\n----- SINGLY LINKED LIST MENU -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Specified Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display Linked List")
    print("10. Search for a Number")
    print("11. Exit")

    choice = int(input("Enter your choice: "))
```

```
    elif choice == 9:
        sll.count_nodes()

    elif choice == 10:
        sll.search()

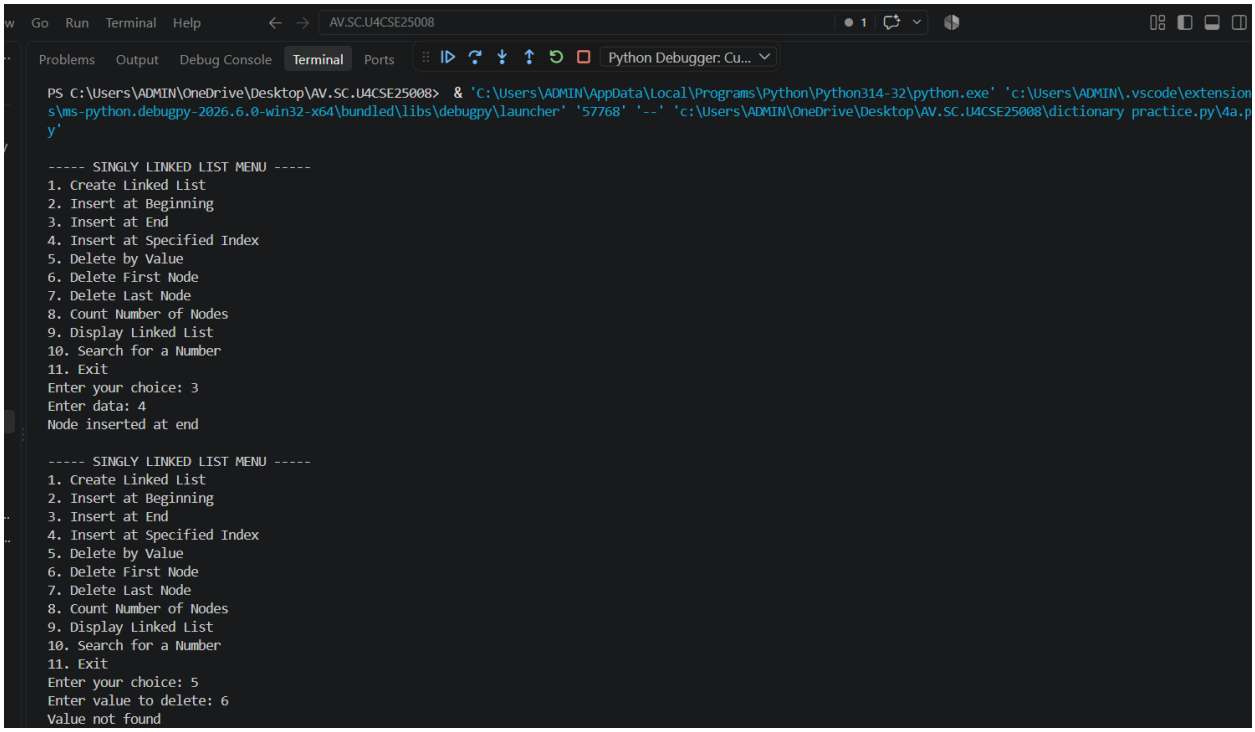
    elif choice == 11:
        print("Exiting from the program...")
        break

    else:
        print("Invalid choice")
```

Output:

```
choice = int(input("Enter your choice: "))

if choice == 1:
    sll.create()
elif choice == 2:
    sll.insert_beginning()
elif choice == 3:
    sll.insert_end()
elif choice == 4:
    sll.insert_index()
elif choice == 5:
    sll.delete_value()
elif choice == 6:
    sll.delete_first()
elif choice == 7:
    sll.delete_last()
elif choice == 8:
    sll.count()
elif choice == 9:
    sll.display()
elif choice == 10:
    sll.search()
elif choice == 11:
    print("Program exited successfully")
    break
else:
    print("Invalid choice! Please try again.")
```



Explanation:

A **Singly Linked List (SLL)** is a linear data structure in which each node contains two parts: **data** and a **next pointer**. The data part stores the value, while the next part stores the address/reference of the next node. The head pointer points to the first node of the linked list. The last node points to None, indicating the end of the list.

The program implements the following operations:

- **Create Linked List:** Creates a linked list by taking the number of nodes and their data from the user. Each new node is connected to the previous node.
- **Insert at Beginning:** Creates a new node and makes it the first node by changing the head pointer.

- **Insert at End:** Traverses the list until the last node and connects the new node to it.
- **Insert at Index:** Inserts a new node at a specified position by adjusting the links between nodes.
- **Delete from Beginning:** Removes the first node by moving the head pointer to the next node.
- **Delete from End:** Traverses to the second-last node and changes its next pointer to None.
- **Delete from Index:** Removes a node from a specified position by changing the next pointer of the previous node.
- **Display:** Traverses the complete linked list and displays each node's data.
- **Search:** Traverses the list and checks each node to find the required element. It also displays its index if found.
- **Count Nodes:** Traverses the list using a temporary pointer and increments count for every node until None is reached.
- **Exit:** Terminates the program.

The program uses a **menu-driven approach**, allowing the user to select and perform different linked-list operations repeatedly.

ERROR TABLE :

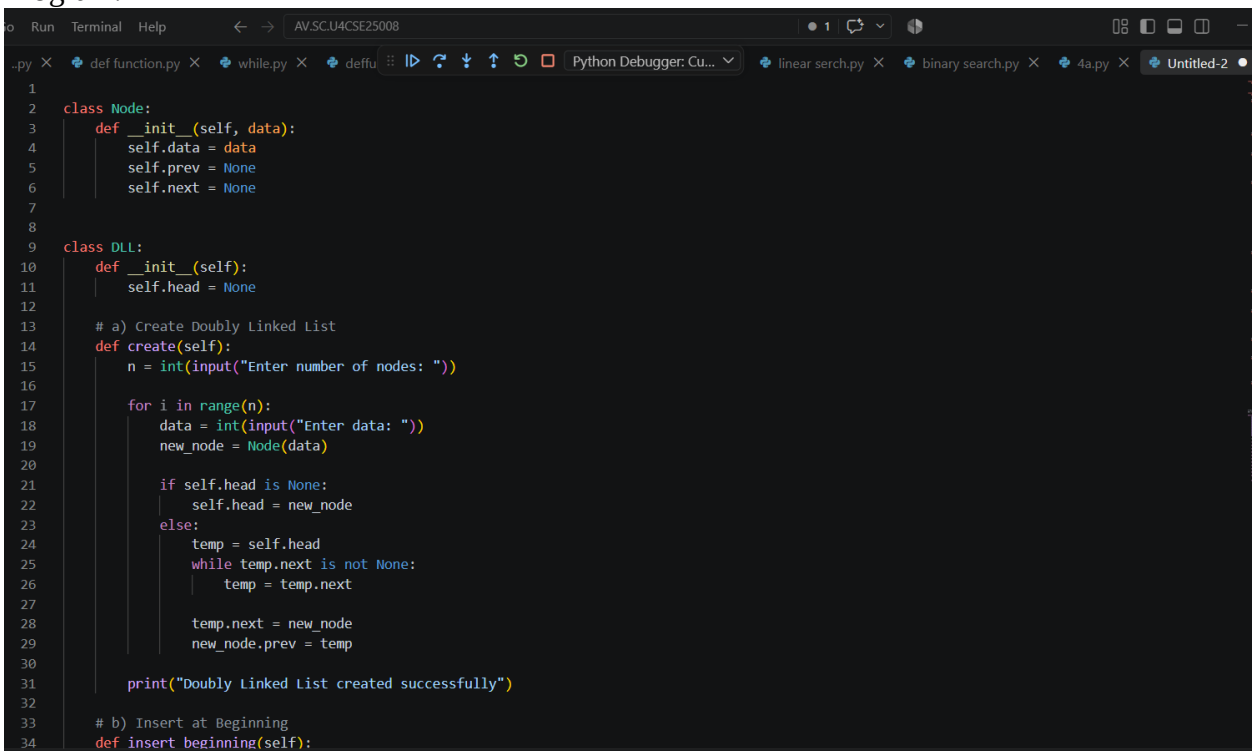
PROGRAM	ERROR / PROBLEM	CAUSE	CORRECTION
1. Create Linked List	List is not created properly	head or next links are not initialized correctly	Initialize head = None and connect each new node using next.
2. Insert at Beginning	New node is not inserted at the beginning	head is not updated correctly	Set new_node.next = self.head and then self.head = new_node.
3. Insert at End	New node is not added at the end	Traversal or next assignment is incorrect	Traverse until temp.next is None and set temp.next = new_node.
4. Insert at Index	Node is inserted at the wrong position	Incorrect index traversal	Traverse up to index - 1 and adjust the next links.
5. Delete from Beginning	First node is not deleted	head is not moved to the next node	Set self.head = self.head.next.
6. Delete from End	Last node is not deleted	Second-last node is not identified correctly	Traverse until temp.next.next is None and set temp.next = None.
7. Delete from Index	Wrong node is deleted	Incorrect index or link modification	Traverse to index - 1 and update temp.next =

			temp.next.next.
8. Display	List elements are not displayed completely	temp is not updated during traversal	Use temp = temp.next until temp is None.
9. Search	Element is not found correctly	List is not traversed completely	Compare each node and update temp = temp.next.
10. Count Nodes	Incorrect number of nodes is displayed	Last node is skipped during counting	Use while temp is not None and increment count for every node.

Aim 4.b:Write a program to implement a double linked list which would perform the below operations, by taking an input from the user.

a)create a linked list
b)insert at beginning
c)insert at end
d)insert at a specified index
e)delete by value
f)delete 1st node
g)delete last node
h)count no.of nodes
i)display or traverse
j)search for a number
k)exit

Program:



```
    data = int(input("Enter data: "))
    new_node = Node(data)

    if self.head is not None:
        new_node.next = self.head
        self.head.prev = new_node

    self.head = new_node
    print("Node inserted at beginning")

# c) Insert at End
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
    else:
        temp = self.head
        while temp.next is not None:
            temp = temp.next

        temp.next = new_node
        new_node.prev = temp

    print("Node inserted at end")

# d) Insert at Specified Index
def insert_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter data: "))
```

```
if index < 0:
    print("Invalid index")
    return

new_node = Node(data)

if index == 0:
    new_node.next = self.head

    if self.head is not None:
        self.head.prev = new_node

    self.head = new_node
    print("Node inserted successfully")
    return

temp = self.head

for i in range(index - 1):
    if temp is None:
        print("Invalid index")
        return
    temp = temp.next

if temp is None:
    print("Invalid index")
    return

new_node.next = temp.next
new_node.prev = temp
```

```
    if temp.next is not None:
        temp.next.prev = new_node

    temp.next = new_node
    print("Node inserted successfully")

# e) Delete by Value
def delete_value(self):
    value = int(input("Enter value to delete: "))
    temp = self.head

    while temp is not None:
        if temp.data == value:

            if temp.prev is not None:
                temp.prev.next = temp.next
            else:
                self.head = temp.next

            if temp.next is not None:
                temp.next.prev = temp.prev

            print("Node deleted successfully")
            return

        temp = temp.next

    print("Value not found")

# f) Delete First Node
def delete_first(self):
```

```
    if self.head is None:
        print("List is empty")
    else:
        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

        print("First node deleted")

# g) Delete Last Node
def delete_last(self):
    if self.head is None:
        print("List is empty")
    else:
        temp = self.head

        if temp.next is None:
            self.head = None
        else:
            while temp.next is not None:
                temp = temp.next

            temp.prev.next = None

        print("Last node deleted")

# h) Count Number of Nodes
def count(self):
    count = 0
    temp = self.head
```

```
    while temp is not None:
        count += 1
        temp = temp.next

    print("Number of nodes:", count)

# i) Display / Traverse
def display(self):
    if self.head is None:
        print("List is empty")
        return

    temp = self.head
    print("Doubly Linked List:", end=" ")

    while temp is not None:
        print(temp.data, end=" <-> ")
        temp = temp.next

    print("None")

# j) Search for a Number
def search(self):
    value = int(input("Enter number to search: "))
    temp = self.head
    position = 0

    while temp is not None:
        if temp.data == value:
            print("Element found at index", position)
            return
```

```
    temp = self.head
    print("Doubly Linked List:", end=" ")

    while temp is not None:
        print(temp.data, end=" <-> ")
        temp = temp.next

    print("None")

# j) Search for a Number
def search(self):
    value = int(input("Enter number to search: "))
    temp = self.head
    position = 0

    while temp is not None:
        if temp.data == value:
            print("Element found at index", position)
            return

        temp = temp.next
        position += 1

    print("Element not found")

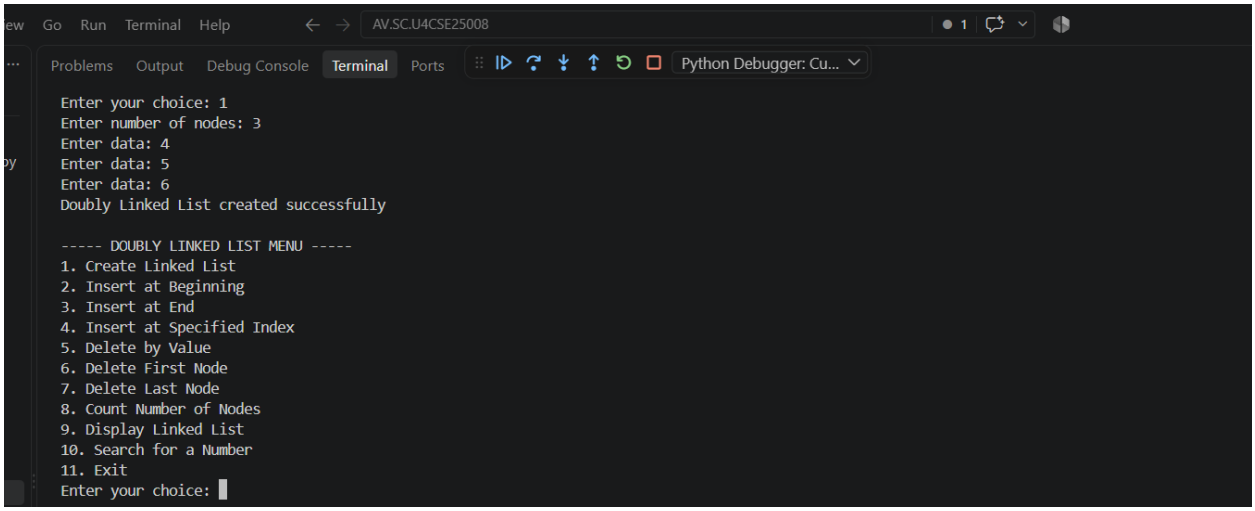
Main Program
ll = DLL()
```

```
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display Linked List")
    print("10. Search for a Number")
    print("11. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()
    elif choice == 2:
        dll.insert_beginning()
    elif choice == 3:
        dll.insert_end()
    elif choice == 4:
        dll.insert_index()
    elif choice == 5:
        dll.delete_value()
    elif choice == 6:
        dll.delete_first()
    elif choice == 7:
        dll.delete_last()
    elif choice == 8:
        dll.count()
    elif choice == 9:
        dll.display()
    elif choice == 10:
        dll.search()
    elif choice == 11:
        print("Program exited successfully")
        break
    else:
```

OUTPUT:



ERROR TABLE:

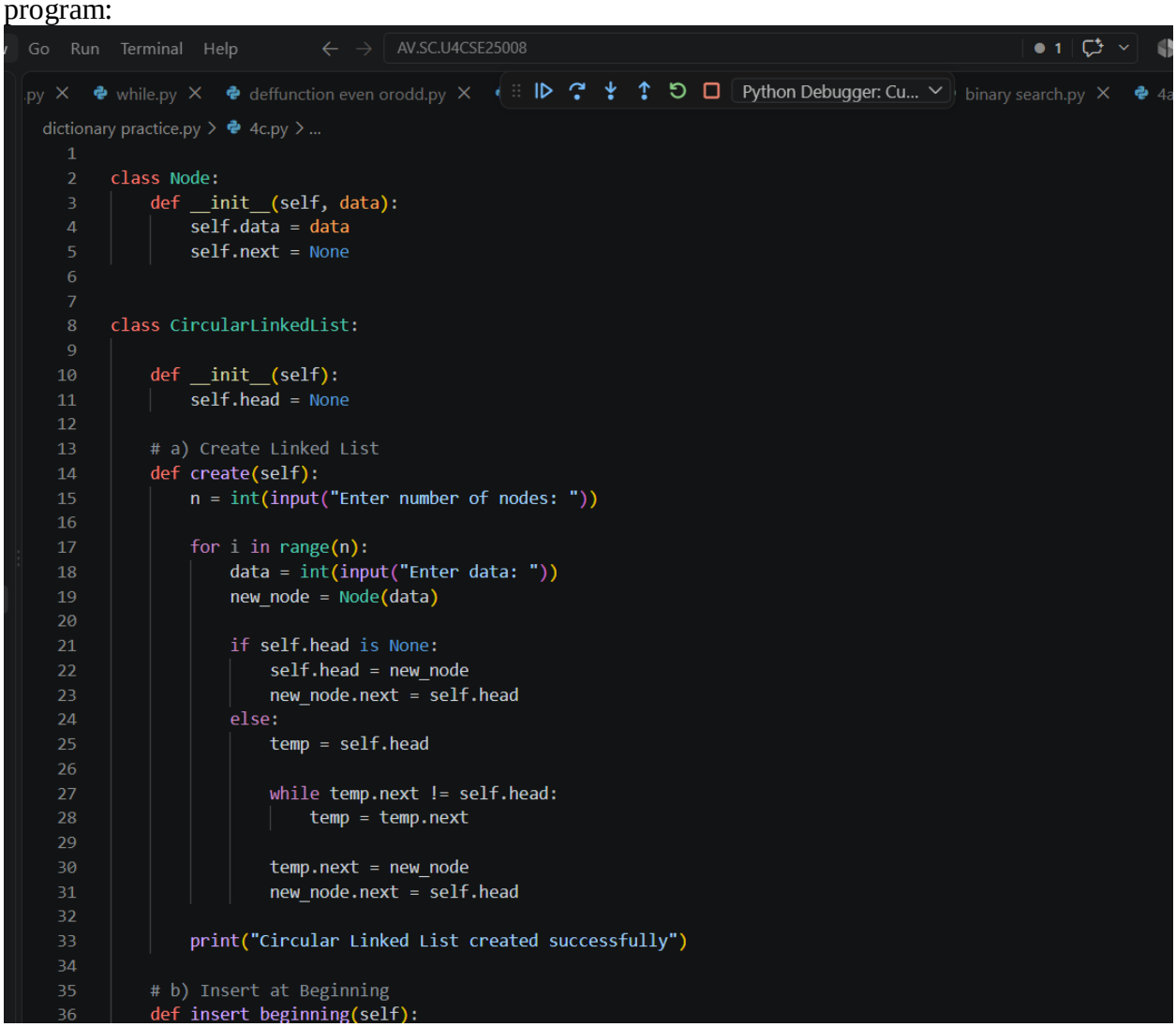
PROGRAM	ERROR / PROBLEM	CAUSE	CORRECTION
1. Create Linked List	List is not created properly	prev or next links are not initialized correctly	Initialize head = None and connect each node using both

			prev and next.
2. Insert at Beginning	New node is not inserted at the beginning	head or prev link is not updated correctly	Set new_node.next = self.head, update the old head.prev, and update self.head.
3. Insert at End	New node is not added at the end	Traversal or next/prev assignment is incorrect	Traverse to the last node and update both next and prev links.
4. Insert at Index	Node is inserted at the wrong position	Incorrect index traversal	Traverse to index - 1 and adjust the prev and next links.
5. Delete by Value	Required node is not deleted	Value is not found or links are not updated	Find the node and reconnect the previous and next nodes correctly.
6. Delete from Beginning	First node is not deleted	head is not moved or prev is not reset	Move head to the next node and set the new head.prev = None.
7. Delete from End	Last node is not deleted	Last or previous node is not identified correctly	Traverse to the last node and set the previous node.next = None.
8. Display	List elements are not displayed completely	Traversal pointer is not updated	Use temp = temp.next until temp is None.
9. Search	Element is not found correctly	List is not traversed completely	Compare each node and update temp = temp.next.
10. Count Nodes	Incorrect number of nodes is displayed	A node is skipped during traversal	Traverse until temp is None and increment count for every node.

Explanation:
A Doubly Linked List (DLL) is a linear data structure in which each node contains three parts: data, a next pointer, and a previous pointer. The next pointer stores the reference to the next node, while the previous pointer stores the reference to the previous node. The head points to the first node, and the previous pointer of the first node is None. The program performs creation, insertion at the

beginning, insertion at the end, insertion at a specified index, deletion, display, search, and counting operations. The menu-driven approach allows the user to repeatedly select the required operation.

Aim 4.c:Write a program to implement a circular linked list which would perform the below operations, by taking an input from the user.
a)create a linked list
b)insert at beginning
c)insert at end
d)insert at a specified index
e)delete by value
f)delete 1st node
g)delete last node
h)count no.of nodes
i)display or traverse
j)search for a number
k)exit
program:



```
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            new_node.next = self.head
            temp.next = new_node
            self.head = new_node

        print("Node inserted at beginning")

    # c) Insert at End
    def insert_end(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = new_node
            new_node.next = self.head
```

```

        print("Node inserted at end")

    # d) Insert at Specified Index
    def insert_index(self):
        index = int(input("Enter index: "))
        data = int(input("Enter data: "))

        if index < 0:
            print("Invalid index")
            return

        new_node = Node(data)

        if index == 0:
            if self.head is None:
                self.head = new_node
                new_node.next = self.head
            else:
                temp = self.head

                while temp.next != self.head:
                    temp = temp.next

                new_node.next = self.head
                temp.next = new_node
                self.head = new_node

            print("Node inserted successfully")
            return

        temp = self.head

        for i in range(index - 1):
```

```
            if temp is None or temp.next == self.head:
                print("Invalid index")
                return
            temp = temp.next

        if temp is None:
            print("Invalid index")
            return

        new_node.next = temp.next
        temp.next = new_node

        print("Node inserted successfully")

    # e) Delete by Value
    def delete_value(self):
        value = int(input("Enter value to delete: "))

        if self.head is None:
            print("List is empty")
            return

        temp = self.head
        prev = None

        while True:
            if temp.data == value:

                if temp == self.head:
                    if self.head.next == self.head:
                        self.head = None
                    else:
                        last = self.head

```

```
                        while last.next != self.head:
                            last = last.next

                        self.head = self.head.next
                        last.next = self.head
                    else:
                        prev.next = temp.next

                    print("Node deleted successfully")
                    return

                prev = temp
                temp = temp.next

                if temp == self.head:
                    break

            print("Value not found")

        # f) Delete First Node
        def delete_first(self):
            if self.head is None:
                print("List is empty")
                return

            if self.head.next == self.head:
                self.head = None
            else:
                temp = self.head

                while temp.next != self.head:
                    temp = temp.next

                self.head = self.head.next
                temp.next = self.head

            print("First node deleted")

        # g) Delete Last Node
        def delete_last(self):
            if self.head is None:
                print("List is empty")
                return

            if self.head.next == self.head:
                self.head = None
            else:
                temp = self.head

                while temp.next.next != self.head:
                    temp = temp.next

                temp.next = self.head

            print("Last node deleted")

        # h) Count Number of Nodes
        def count(self):
            if self.head is None:
                print("Number of nodes: 0")
                return

            count = 0
            temp = self.head

            while True:
                count += 1
                temp = temp.next
```

```
            if temp == self.head:
                break

        print("Number of nodes:", count)

    # i) Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head
        print("Circular Linked List:", end=" ")

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next

            if temp == self.head:
                break

        print("(Back to Head)")

    # j) Search for a Number
    def search(self):
        value = int(input("Enter number to search: "))

        if self.head is None:
            print("List is empty")
            return

        temp = self.head
        position = 0
```

Click to add a breakpoint

```
        while True:
            if temp.data == value:
                print("Element found at index", position)
                return

            temp = temp.next
            position += 1

            if temp == self.head:
                break

        print("Element not found")


# Main Program
cll = CircularLinkedList()

while True:
    print("\n----- CIRCULAR LINKED LIST MENU -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Specified Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display Linked List")
    print("10. Search for a Number")
    print("11. Exit")

    choice = int(input("Enter your choice: "))
```

```
272
273     choice = int(input("Enter your choice: "))
274
275     if choice == 1:
276         cll.create()
277     elif choice == 2:
278         cll.insert_beginning()
279     elif choice == 3:
280         cll.insert_end()
281     elif choice == 4:
282         cll.insert_index()
283     elif choice == 5:
284         cll.delete_value()
285     elif choice == 6:
286         cll.delete_first()
287     elif choice == 7:
288         cll.delete_last()
289     elif choice == 8:
290         cll.count()
291     elif choice == 9:
292         cll.display()
293     elif choice == 10:
294         cll.search()
295     elif choice == 11:
296         print("Program exited successfully")
297         break
298     else:
299         print("Invalid choice! Please try again.")
300
```

output:



TIME COMPLEXITY AND SPACE COMPLEXITY

PROGRAM	BEST TIME	AVERAGE TIME	WORST TIME	SPACE COMPLEXITY	REASON
1. Create	O(n)	$O(n^2)$	$O(n^2)$	O(n)	Each node

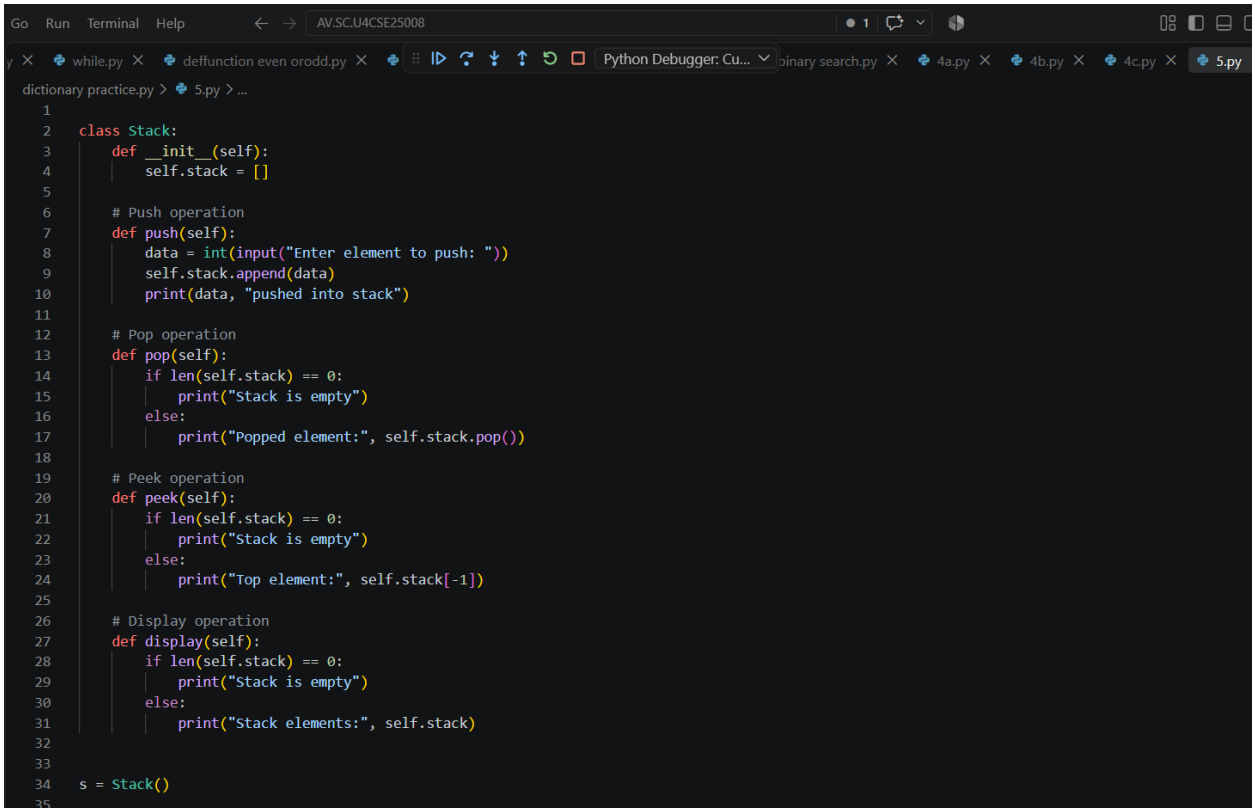
Linked List					is created and connected. Traversing to the end for every insertion makes creation $O(n^2)$.
2. Insert at Beginning	O(1)	O(1)	O(1)	O(1)	A new node is directly connected to the current head.
3. Insert at End	O(n)	O(n)	O(n)	O(1)	The list is traversed until the last node before insertion.
4. Insert at Index	O(1)	O(n)	O(n)	O(1)	The list may need to be traversed up to the required index.
5. Delete from Beginning	O(1)	O(1)	O(1)	O(1)	Only the head pointer is moved to the next node.
6. Delete from End	O(n)	O(n)	O(n)	O(1)	The list must be traversed to find the second-last node.
7. Delete from Index	O(1)	O(n)	O(n)	O(1)	Traversal may be required to reach the node before the specified index.
8. Display	O(n)	O(n)	O(n)	O(1)	Every node is visited once to display its data.
9. Search	O(1)	O(n)	O(n)	O(1)	The element

					may be found at the first node or require checking the entire list.
10. Count Nodes	O(n)	O(n)	O(n)	O(1)	Every node is visited once to calculate the total count.

LAB 5

5. AIM: Write a python program to implement a stack using an Array, Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.

PROGRAM:



```python

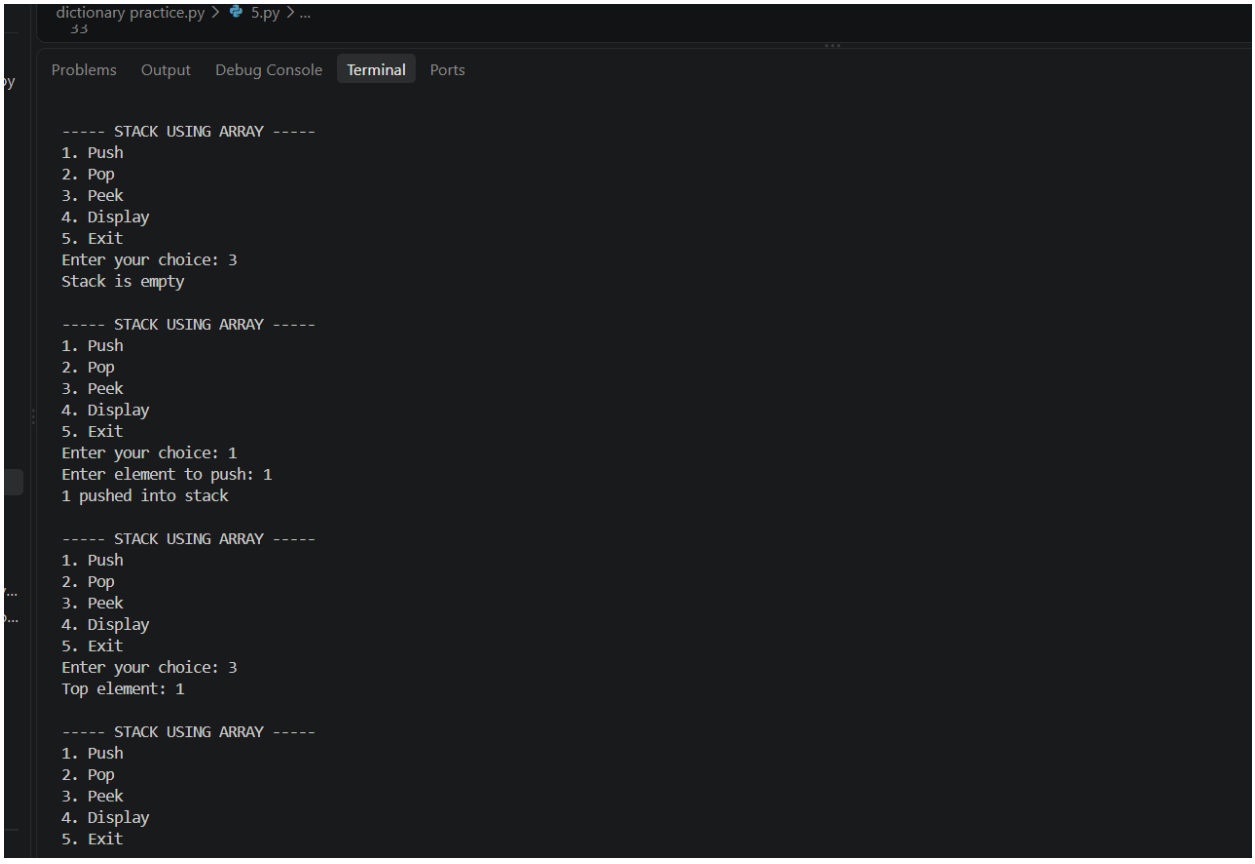
while True:
    print("\n----- STACK USING ARRAY -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        s.push()
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Program exited successfully")
        break
    else:
        print("Invalid choice")

```

Output:



EXPLANATION:

Stack Using Array

Structure:

- Uses a Python list (self.stack) to store elements.
- self.top keeps track of the index of the last pushed element.
- self.size defines the maximum capacity.

Operations:

1. Push
    - Check if top == size - 1 → Overflow.
    - Otherwise, increment top and insert the element at that position.
2. Pop
    - Check if top == -1 → Underflow (empty stack).
    - Otherwise, remove the element at top, decrement top.
3. Peek
    - Shows the element at top without removing it.
4. Display
    - Prints elements from top down to 0 (top to bottom order)

Stack Using Linked List

Structure:

- Each element is a Node with:
    - data (value)
    - next (pointer to the next node).
- self.top points to the most recently pushed node.

Operations:

1. Push
    - Create a new node.
    - Link its next to the current top.
    - Update top to this new node.
2. Pop
    - If top is None → Underflow.

- Otherwise, remove the node at top and update top to top.next.

3. Peek
    - Shows the data at top.
4. Display
    - Traverse from top downwards, printing each node's data.

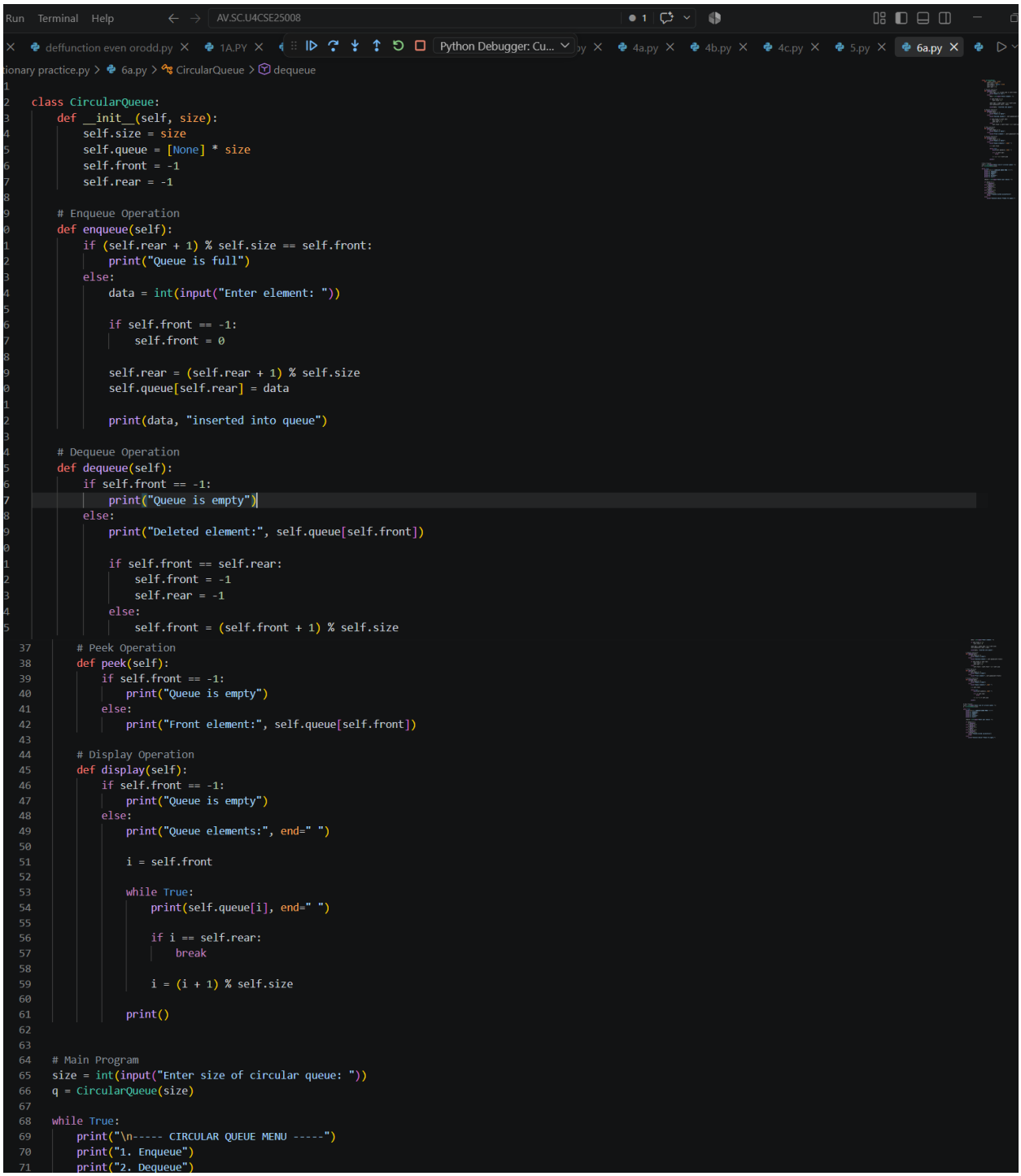
ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	def _init_(self, size):	Python expects **two underscores** before and after init.
2.	Display on empty stack.	Add condition: if self.top is None: → print "Stack is empty".
3.	Stack Overflow when pushing beyond the defined size	Add a check: if self.top == self.size - 1: → print "Stack Overflow".
4.	Stack Underflow when popping from empty stack	Add a check: if self.top is None: → print "Stack Underflow".
5.	Syntax error	Later it was rectified in the for loop of display section.

Lab 6

Aim : Write a python program to implement a queue using an Array, Linked list, circular queue. Implement the queue operations such as enqueue, dequeue, peak and also display the queue elements.

Program: Circular Queue


Run Terminal Help
AV.SC.U4CSE25008
deffunction even orodd.py
1A.PY
Python Debugger: Cu...
4a.py
4b.py
4c.py
5.py
6a.py
ionary practice.py > 6a.py > CircularQueue > dequeue
class CircularQueue:
def __init__(self, size):
self.size = size
self.queue = [None] * size
self.front = -1
self.rear = -1
Enqueue Operation
def enqueue(self):
if (self.rear + 1) % self.size == self.front:
print("Queue is full")
else:
data = int(input("Enter element: "))
if self.front == -1:
self.front = 0
self.rear = (self.rear + 1) % self.size
self.queue[self.rear] = data
print(data, "inserted into queue")
Dequeue Operation
def dequeue(self):
if self.front == -1:
print("Queue is empty")
else:
print("Deleted element:", self.queue[self.front])
if self.front == self.rear:
self.front = -1
self.rear = -1
else:
self.front = (self.front + 1) % self.size
Peek Operation
def peek(self):
if self.front == -1:
print("Queue is empty")
else:
print("Front element:", self.queue[self.front])
Display Operation
def display(self):
if self.front == -1:
print("Queue is empty")
else:
print("Queue elements:", end=" ")
i = self.front
while True:
print(self.queue[i], end=" ")
if i == self.rear:
break
i = (i + 1) % self.size
print()
Main Program
size = int(input("Enter size of circular queue: "))
q = CircularQueue(size)
while True:
print("\n----- CIRCULAR QUEUE MENU -----")
print("1. Enqueue")
print("2. Dequeue")

```
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        q.enqueue()
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program exited successfully")
        break
    else:
        print("Invalid choice! Please try again.")

```

Output:

```
Problems  Output  Debug Console  Terminal  Ports

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Queue is empty

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Queue is empty

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
```

Explanation:

Queue Using Array

Structure:

- Uses a Python list self.queue to store elements.
- self.front keeps track of the **first element** in the queue.
- self.rear keeps track of the **last element** in the queue.
- self.size defines the **maximum capacity** of the queue.
- Initially:

- front = -1
- rear = -1

Operations:

1. Enqueue

- Check if rear == size - 1 → **Overflow** (queue is full).
- If the queue is empty, set front = 0.
- Increment rear.
- Insert the element at rear.

2. Dequeue

- Check if front == -1 or front > rear → **Underflow** (queue is empty).
- Remove the element at front.
- Increment front.
- If the queue becomes empty, reset front and rear to -1.

3. Peek

- Shows the element at front without removing it.
- If the queue is empty → Underflow/empty message.

4. Display

- Prints elements from front to rear.
- Elements are displayed in **front-to-rear order**.

Important Points:

- Queue follows **FIFO (First In, First Out)**.
- Insertion takes place at the **rear**.
- Deletion takes place at the **front**.
- **Overflow** → queue is full.
- **Underflow** → queue is empty.

A0B

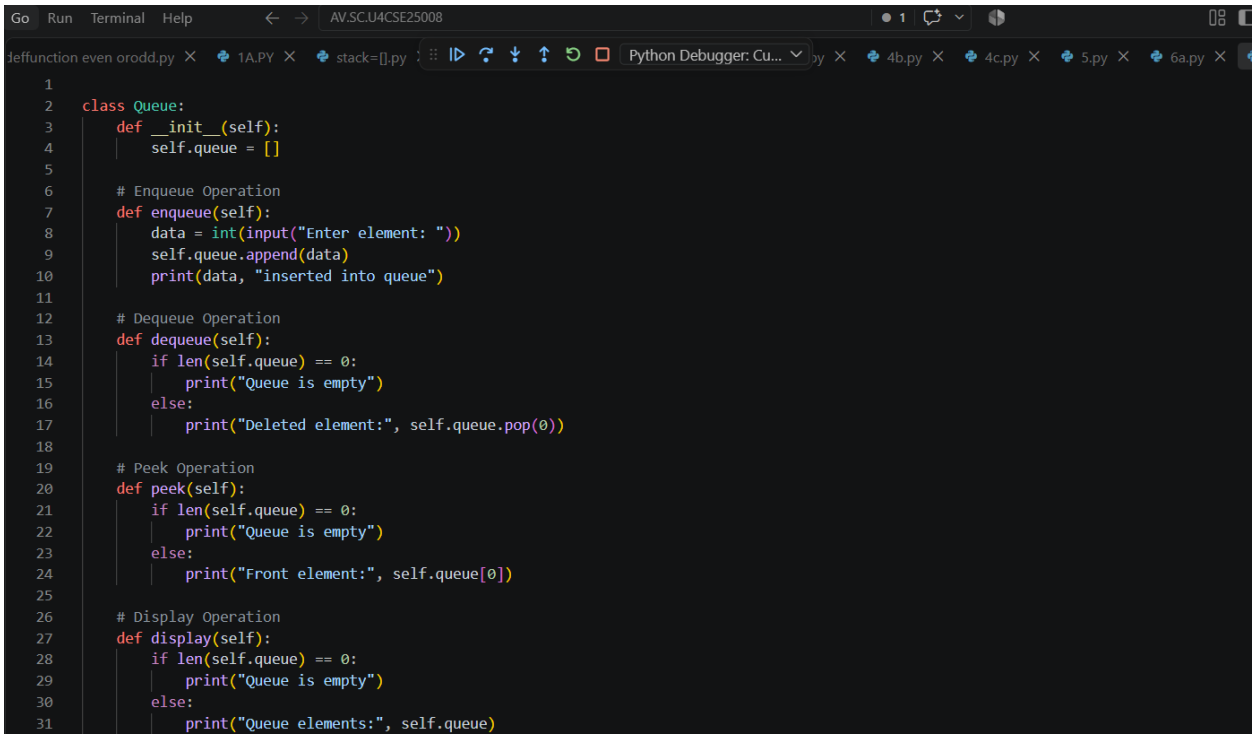
Error table:

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Display on empty circular queue.	Add condition: if self.front == -1: → print "Queue is empty".
2.	Circular Queue Overflow	Add a check: if (self.rear + 1) % self.size == self.front: → print "Circular Queue Overflow".

3.	Circular Queue Underflow	Add a check: if self.front == -1: → print "Circular Queue Underflow".
4.	rear does not wrap around to the beginning of the array.	Update using: self.rear = (self.rear + 1) % self.size.

Program:Array




```python

class Queue:
    def __init__(self):
        self.queue = []

    # Enqueue Operation
    def enqueue(self):
        data = int(input("Enter element: "))
        self.queue.append(data)
        print(data, "inserted into queue")

    # Dequeue Operation
    def dequeue(self):
        if len(self.queue) == 0:
            print("Queue is empty")
        else:
            print("Deleted element:", self.queue.pop(0))

    # Peek Operation
    def peek(self):
        if len(self.queue) == 0:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[0])

    # Display Operation
    def display(self):
        if len(self.queue) == 0:
            print("Queue is empty")
        else:
            print("Queue elements:", self.queue)
```

```python

# Main Program
q = Queue()

while True:
    print("\n----- QUEUE USING ARRAY -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        q.enqueue()
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program exited successfully")
        break
    else:
        print("Invalid choice! Please try again.")

```

Output:

Positive case:

```
Problems  Output  Debug Console  Terminal  Ports

y'

----- QUEUE USING ARRAY -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Queue is empty

----- QUEUE USING ARRAY -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Queue is empty

----- QUEUE USING ARRAY -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
```

Explanation:

Queue Using Linked List

Structure:

- Each element is represented using a **Node**.
- Each Node contains:
    - data → stores the value.

    - next → points to the next node.
- self.front points to the **first node**.
- self.rear points to the **last node**.
- Initially:
    - front = None
    - rear = None

Operations:

1. Enqueue

- Create a new node.
- If the queue is empty:
    - Set both front and rear to the new node.
- Otherwise:
    - Link rear.next to the new node.
    - Update rear to the new node.

2. Dequeue

- Check if front is None → **Underflow**.
- Store/remove the node at front.
- Move front to front.next.
- If front becomes None, set rear = None.

3. Peek

- Shows the data stored at front without removing it.
- If front is None, the queue is empty.

4. Display

- Start from front.
- Traverse using next.
- Print each element until None.

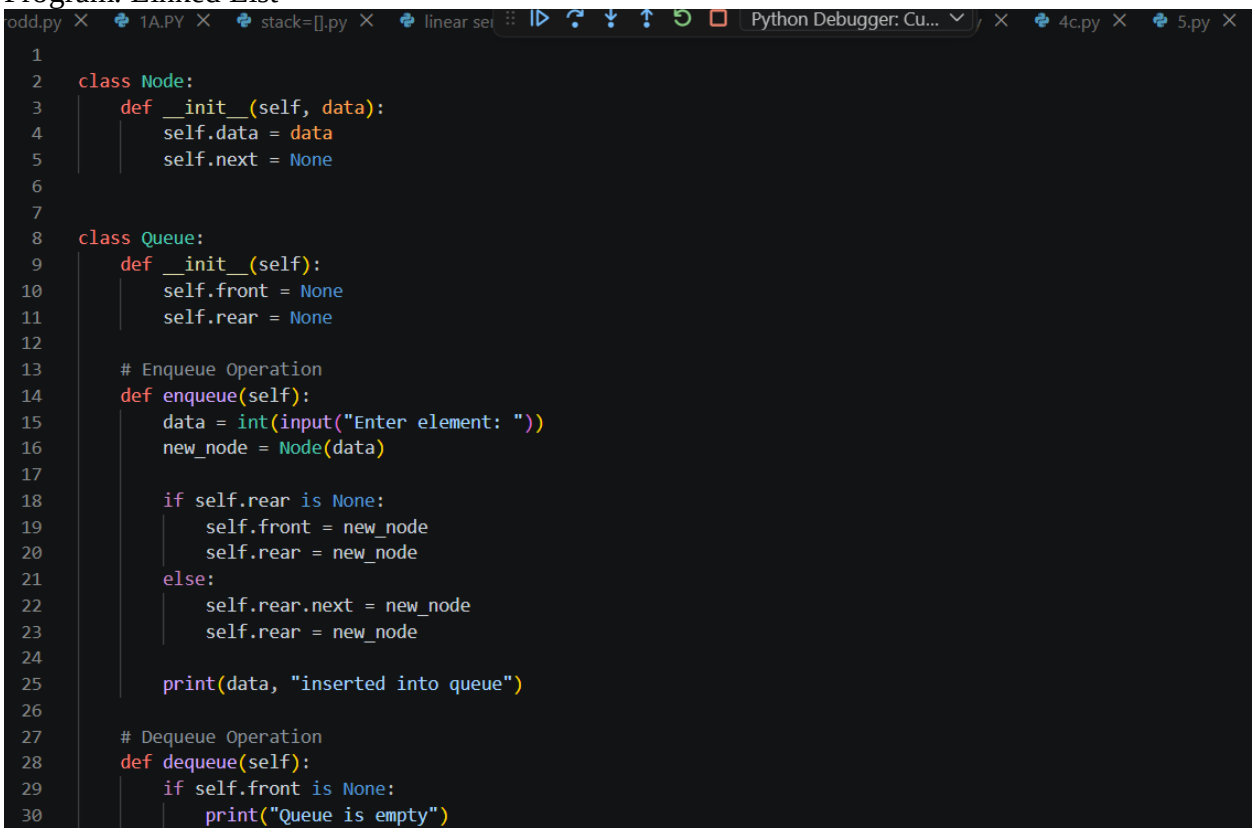
Important Points:

- Follows **FIFO**.
- Insertion → **rear**.
- Deletion → **front**.
- No fixed size is required.
- Memory is allocated dynamically.

ERROR TABLE: QUEUE USING LINKED LIST

S.NO	Error Obtained	Rectification
1.	Display on empty queue.	Add condition: if self.front is None: → print "Queue is empty".
2.	Queue Underflow when deleting from an empty queue.	Add a check: if self.front is None: → print "Queue Underflow".
3.	First node inserted without updating front and rear.	If the queue is empty, set self.front = self.rear = new.
4.	rear not updated after insertion.	Update using self.rear = new after linking the new node.

Program: Linked List



Output:



Explanation:

Circular Queue Using Array

Structure:

- Uses a Python list self.queue to store elements.
- self.front points to the **first element**.
- self.rear points to the **last element**.
- self.size defines the maximum capacity.
- When rear reaches the last index, it can **wrap around to index 0**.
- This allows previously freed positions to be reused.

Operations:

1. Enqueue

- Check if (rear + 1) % size == front → **Overflow** (circular queue is full).
- If the queue is empty:
  - Set front = 0
  - Set rear = 0
- Otherwise:
  - Move rear using:
    rear = (rear + 1) % size
- Insert the element at rear.

2. Dequeue

- Check if front == -1 → **Underflow**.
- Remove the element at front.
- If front == rear:
  - The queue becomes empty.
  - Set front = -1 and rear = -1.

- Otherwise:
    - Move front using:
      front = (front + 1) % size

3. Peek

- Shows the element at front without removing it.
- If front == -1, the queue is empty.

4. Display

- Start from front.
- Move towards rear.
- Use modulo % size to wrap around to the beginning of the array.

Important Points:

- Follows **FIFO**.
- Insertion → **rear**.
- Deletion → **front**.
- The last position connects logically back to the first position.
- **Unused spaces can be reused**, unlike a simple linear array queue.
- Overflow occurs when the next rear position becomes the front.

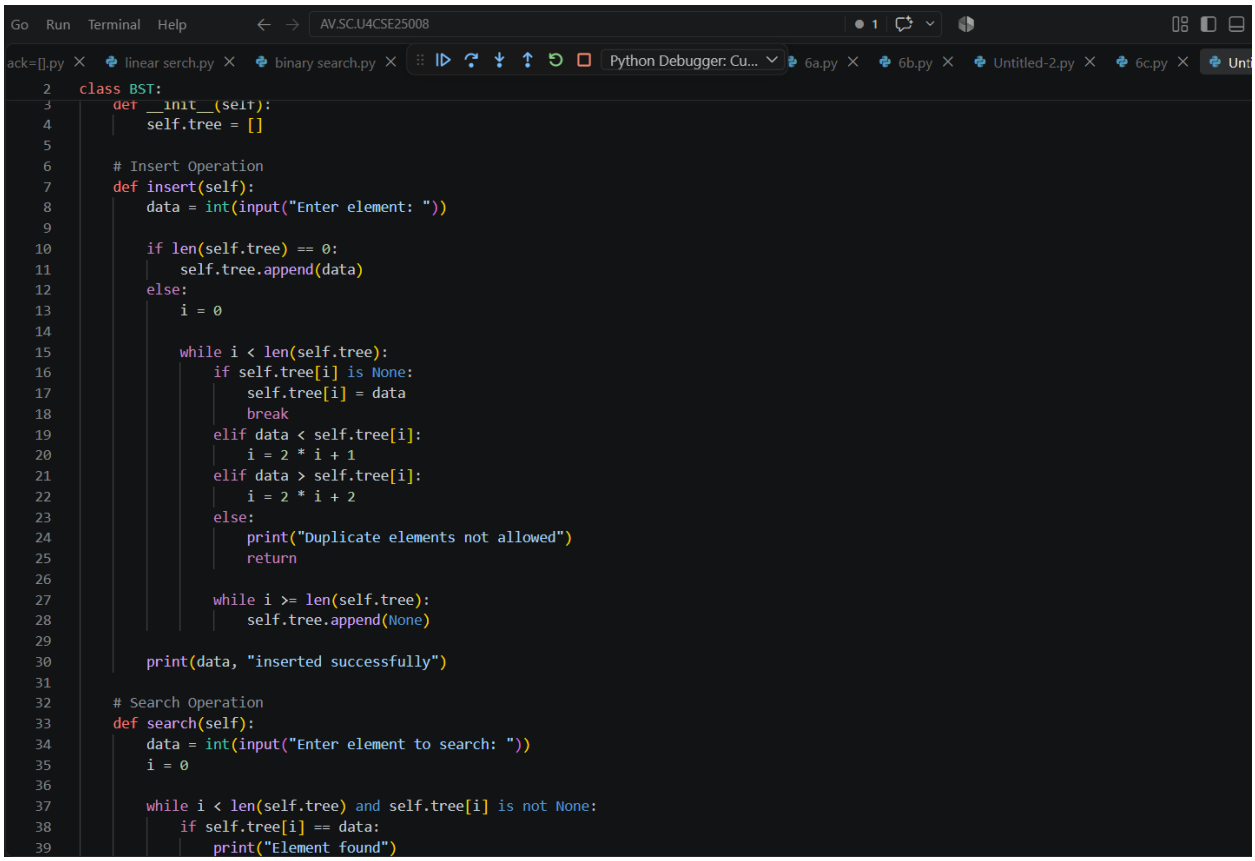
Error Table: **QUEUE USING ARRAY**

S.NO	Error Obtained	Rectification
1.	Display on empty queue.	Add condition: if self.front == -1: → print "Queue is empty".
2.	Queue Overflow when inserting beyond the defined size.	Add a check: if self.rear == self.size - 1: → print "Queue Overflow".
3.	Queue Underflow when deleting from an empty queue.	Add a check: if self.front == -1 or self.front > self.rear: → print "Queue Underflow".
4.	First element inserted without setting front.	If the queue is empty, set self.front = 0 before increasing rear.

Lab 7

AIM a)write a python program to implement binary search tree using an array with all its operations.

Program:



```
Go  Run  Terminal  Help   AV.SC.U4CSE25008   1
ack=[].py  linear serch.py  binary search.py  Python Debugger: Cu...  6a.py  6b.py  Untitled-2.py  6c.py  Unt
```

```python
class BST:
    def __init__(self):
        self.tree = []

    # Insert Operation
    def insert(self):
        data = int(input("Enter element: "))

        if len(self.tree) == 0:
            self.tree.append(data)
        else:
            i = 0

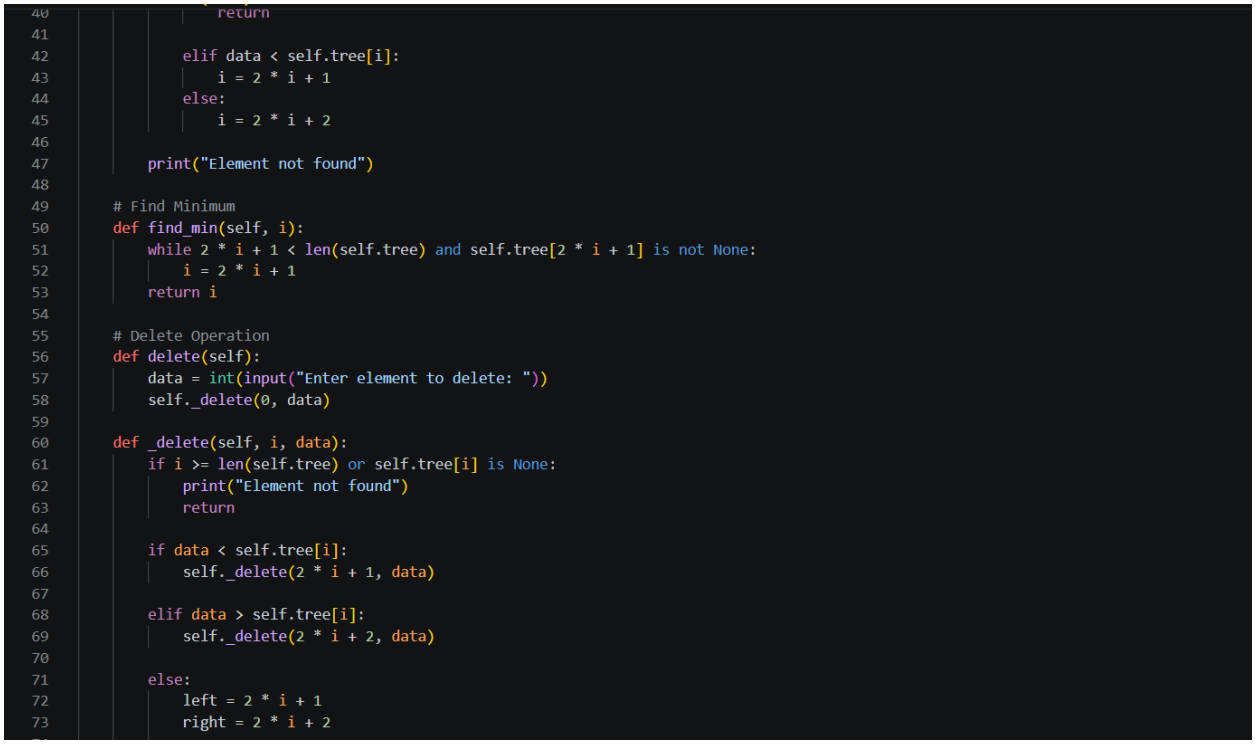
            while i < len(self.tree):
                if self.tree[i] is None:
                    self.tree[i] = data
                    break
                elif data < self.tree[i]:
                    i = 2 * i + 1
                elif data > self.tree[i]:
                    i = 2 * i + 2
                else:
                    print("Duplicate elements not allowed")
                    return

                while i >= len(self.tree):
                    self.tree.append(None)

        print(data, "inserted successfully")

    # Search Operation
    def search(self):
        data = int(input("Enter element to search: "))
        i = 0

        while i < len(self.tree) and self.tree[i] is not None:
            if self.tree[i] == data:
                print("Element found")
```



```python
                return

            elif data < self.tree[i]:
                i = 2 * i + 1
            else:
                i = 2 * i + 2

        print("Element not found")

    # Find Minimum
    def find_min(self, i):
        while 2 * i + 1 < len(self.tree) and self.tree[2 * i + 1] is not None:
            i = 2 * i + 1
        return i

    # Delete Operation
    def delete(self):
        data = int(input("Enter element to delete: "))
        self._delete(0, data)

    def _delete(self, i, data):
        if i >= len(self.tree) or self.tree[i] is None:
            print("Element not found")
            return

        if data < self.tree[i]:
            self._delete(2 * i + 1, data)

        elif data > self.tree[i]:
            self._delete(2 * i + 2, data)

        else:
            left = 2 * i + 1
            right = 2 * i + 2
```

```
            if (left >= len(self.tree) or self.tree[left] is None) and \
               (right >= len(self.tree) or self.tree[right] is None):
                self.tree[i] = None

            elif left >= len(self.tree) or self.tree[left] is None:
                self.tree[i] = self.tree[right]
                self._delete(right, self.tree[right])

            elif right >= len(self.tree) or self.tree[right] is None:
                self.tree[i] = self.tree[left]
                self._delete(left, self.tree[left])

            else:
                min_index = self.find_min(right)
                self.tree[i] = self.tree[min_index]
                self._delete(min_index, self.tree[min_index])

            print("Element deleted successfully")

    # Inorder Traversal
    def inorder(self, i=0):
        if i < len(self.tree) and self.tree[i] is not None:
            self.inorder(2 * i + 1)
            print(self.tree[i], end=" ")
            self.inorder(2 * i + 2)

    # Preorder Traversal
    def preorder(self, i=0):
        if i < len(self.tree) and self.tree[i] is not None:
            print(self.tree[i], end=" ")
            self.preorder(2 * i + 1)
            self.preorder(2 * i + 2)
```

```

    # Postorder Traversal
    def postorder(self, i=0):
        if i < len(self.tree) and self.tree[i] is not None:
            self.postorder(2 * i + 1)
            self.postorder(2 * i + 2)
            print(self.tree[i], end=" ")

    # Display Array
    def display(self):
        print("BST Array:", self.tree)


# Main Program
bst = BST()

while True:
    print("\n----- BINARY SEARCH TREE USING ARRAY -----")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Display")
    print("8. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        bst.insert()
    elif choice == 2:
        bst.search()
    elif choice == 3:
        bst.delete()
```

```
142     elif choice == 4:
143         print("Inorder:", end=" ")
144         bst.inorder()
145         print()
146     elif choice == 5:
147         print("Preorder:", end=" ")
148         bst.preorder()
149         print()
150     elif choice == 6:
151         print("Postorder:", end=" ")
152         bst.postorder()
153         print()
154     elif choice == 7:
155         bst.display()
156     elif choice == 8:
157         print("Program exited successfully")
158         break
159     else:
160         print("Invalid choice! Please try again.")
161
```

output:

```
----- BINARY SEARCH TREE USING ARRAY -----
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice: 3
Enter element to delete: 4
Element not found

----- BINARY SEARCH TREE USING ARRAY -----
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display
8. Exit
Enter your choice:
```

Explanation:

Binary Search Tree (BST) Using Array

Structure:

- Uses a Python list self.tree to store the elements.
- The root element is stored at **index 0**.
- For a node at index i:
  - **Left child** → 2*i + 1
  - **Right child** → 2*i + 2

- An empty position is represented using None.
- BST follows the rule:
    - Left subtree contains **smaller** elements.
    - Right subtree contains **greater** elements.

Operations:

1. Insert

- Start from index 0 (root).
- Compare the new element with the current element.
- If the element is smaller → move to 2*i + 1.
- If the element is greater → move to 2*i + 2.
- Continue until an empty position (None) is found.
- Insert the element at that position.

2. Search

- Start from the root.
- Compare the required element with the current element.
- If equal → element is found.
- If smaller → move to the left child.
- If greater → move to the right child.
- If None is reached → element is not present.

3. Delete

There are three cases:

- **No child:** Set that position to None.
- **One child:** Replace the node with its child.
- **Two children:** Find the minimum element in the right subtree (inorder successor), replace the deleted element with it, and remove the original successor.

4. Minimum

- Start from the given node.
- Keep moving to the left child.
- The leftmost element is the minimum value.

5. Maximum

- Start from the given node.
- Keep moving to the right child.
- The rightmost element is the maximum value.

6. Inorder Traversal

- Traverse **Left → Root → Right**.
- Inorder traversal of a BST gives elements in **sorted order**.

7. Preorder Traversal

- Traverse **Root → Left → Right**.

8. Postorder Traversal

- Traverse **Left → Right → Root**.

9. Count

- Counts the total number of nodes in the BST.
- Formula:
  Count = 1 + left subtree count + right subtree count

10. Height

- Finds the longest path from the root to a leaf.
- Formula:
  Height = 1 + max(left height, right height)

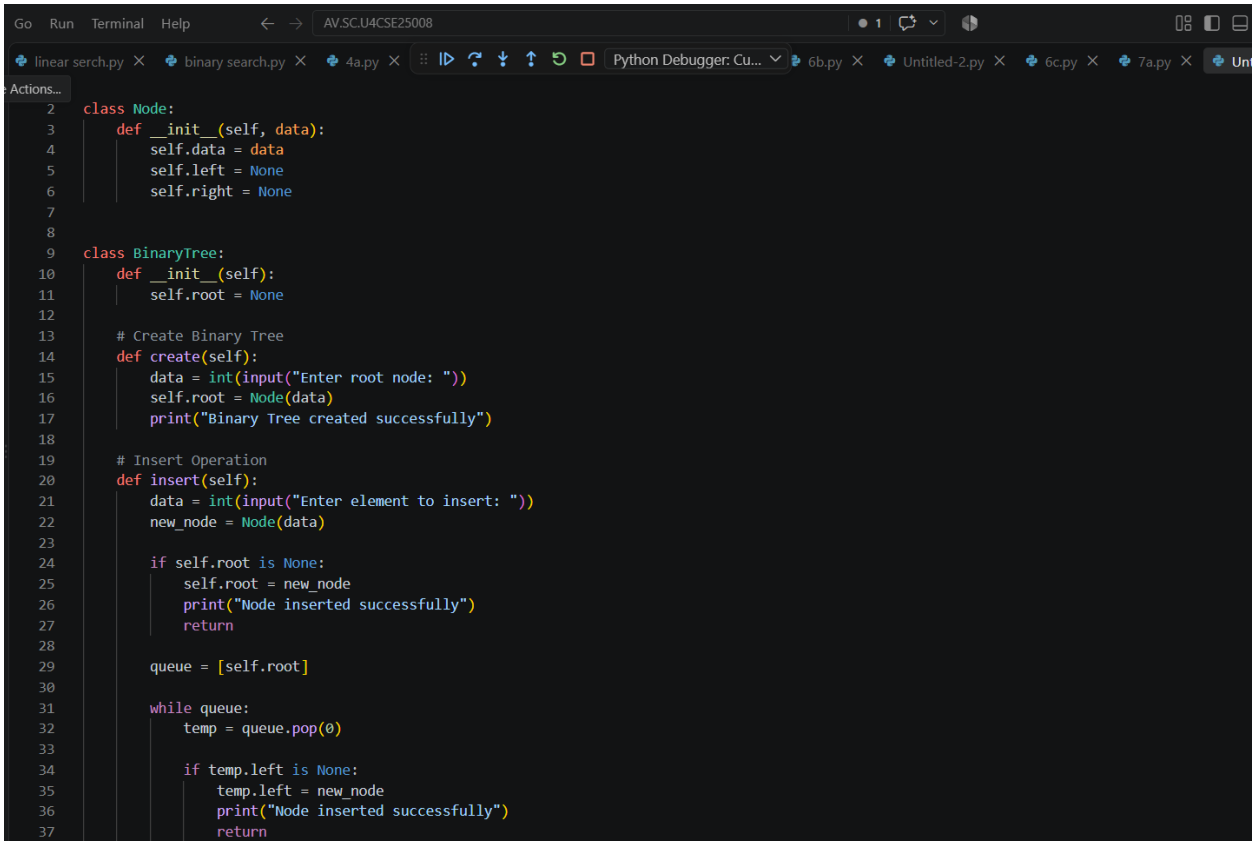
Important Points:

- Root is at index 0.
- Left child → 2*i + 1.
- Right child → 2*i + 2.
- None represents an empty position.
- Inorder traversal gives **sorted elements**.
- BST provides efficient searching when the tree is balanced.

Error Table:

S.NO	Error Obtained	Rectification
1.	Inserting into an occupied position without comparing the value.	Compare data with self.tree[i]. If data is smaller, move to 2*i + 1; otherwise move to 2*i + 2.
2.	Searching beyond the array limit.	Check the index condition: while i < 100 and self.tree[i] is not None.
3.	Searching for a value that does not exist.	If self.tree[i] is None, return False.
4.	Incorrect child index calculation.	Use left child = 2*i + 1 and right child = 2*i + 2.

AIM b) write a python program to implement binary tree using an linked list with all its operations.



```python
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinaryTree:
    def __init__(self):
        self.root = None

    # Create Binary Tree
    def create(self):
        data = int(input("Enter root node: "))
        self.root = Node(data)
        print("Binary Tree created successfully")

    # Insert Operation
    def insert(self):
        data = int(input("Enter element to insert: "))
        new_node = Node(data)

        if self.root is None:
            self.root = new_node
            print("Node inserted successfully")
            return

        queue = [self.root]

        while queue:
            temp = queue.pop(0)

            if temp.left is None:
                temp.left = new_node
                print("Node inserted successfully")
                return
            else:
                queue.append(temp.left)

            if temp.right is None:
                temp.right = new_node
                print("Node inserted successfully")
                return
            else:
                queue.append(temp.right)

    # Search Operation
    def search(self):
        data = int(input("Enter element to search: "))

        if self.root is None:
            print("Tree is empty")
            return

        queue = [self.root]

        while queue:
            temp = queue.pop(0)

            if temp.data == data:
                print("Element found")
                return

            if temp.left:
                queue.append(temp.left)

            if temp.right:
                queue.append(temp.right)

        print("Element not found")
```

```
    # Delete Operation
    def delete(self):
        data = int(input("Enter element to delete: "))

        if self.root is None:
            print("Tree is empty")
            return

        queue = [(self.root, None)]
        target = None
        last = None
        parent = None

        while queue:
            temp, par = queue.pop(0)
            last = temp
            parent = par

            if temp.data == data and target is None:
                target = temp

            if temp.left:
                queue.append((temp.left, temp))

            if temp.right:
                queue.append((temp.right, temp))

        if target is None:
            print("Element not found")
            return

        if last == target:
            if parent is None:
                self.root = None
            elif parent.left == last:
                parent.left = None
            else:
                parent.right = None
        else:
            target.data = last.data

            if parent.left == last:
                parent.left = None
            else:
                parent.right = None

        print("Node deleted successfully")

    # Inorder Traversal
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    # Preorder Traversal
    def preorder(self, root):
        if root:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    # Postorder Traversal
    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")
```

```
    # Display Binary Tree
    def display(self):
        if self.root is None:
            print("Tree is empty")
        else:
            print("Binary Tree elements (Level Order):")
            queue = [self.root]

            while queue:
                temp = queue.pop(0)
                print(temp.data, end=" ")

                if temp.left:
                    queue.append(temp.left)

                if temp.right:
                    queue.append(temp.right)

            print()


# Main Program
bt = BinaryTree()

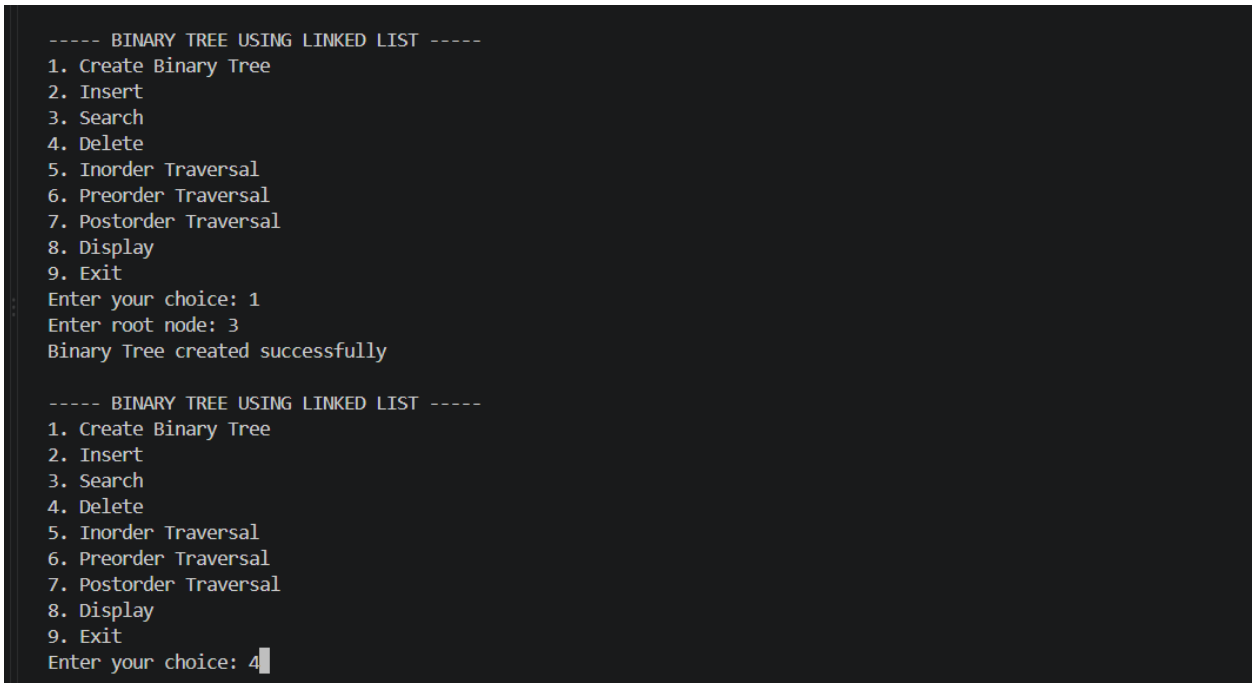
while True:
    print("\n----- BINARY TREE USING LINKED LIST -----")
    print("1. Create Binary Tree")
    print("2. Insert")
    print("3. Search")
    print("4. Delete")
    print("5. Inorder Traversal")
    print("6. Preorder Traversal")
    print("7. Postorder Traversal")
    print("8. Display")
    print("9. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        bt.create()
    elif choice == 2:
        bt.insert()
    elif choice == 3:
        bt.search()
    elif choice == 4:
        bt.delete()
    elif choice == 5:
        print("Inorder:", end=" ")
        bt.inorder(bt.root)
        print()
    elif choice == 6:
        print("Preorder:", end=" ")
        bt.preorder(bt.root)
        print()
    elif choice == 7:
        print("Postorder:", end=" ")
        bt.postorder(bt.root)
        print()
    elif choice == 8:
        bt.display()
    elif choice == 9:
        print("Program exited successfully")
        break
    else:
        print("Invalid choice! Please try again.")

```

OUTPUT:



Explanation:

Binary Search Tree (BST) Using Linked List

Structure:

- Each element is represented using a **Node**.
- Each Node contains:
  - data → stores the value.
  - left → points to the left child.
  - right → points to the right child.
- self.root points to the **root node** of the BST.
- Initially, root = None, which means the tree is empty.
- BST follows the rule:
  - Left subtree contains **smaller** elements.
  - Right subtree contains **greater** elements.

Operations:

1. Insert

- Create a new node.
- If the tree is empty, make the new node the root.
- Otherwise, start from the root.
- If the new element is smaller → move to the left.
- If the new element is greater → move to the right.
- Continue until an empty position is found.

- Link the new node at that position.

2. Search
- Start from the root.
- Compare the required element with the current node.
- If equal → element is found.
- If smaller → move to the left child.
- If greater → move to the right child.
- If None is reached → element is not present.

3. Delete
There are three cases:
- **No child:** Remove the node.
- **One child:** Replace the node with its child.
- **Two children:** Find the minimum element in the right subtree (inorder successor), copy its value into the node, and delete the original successor node.

4. Minimum
- Start from the given node.
- Keep moving to the left child.
- The leftmost node contains the minimum value.

5. Maximum
- Start from the given node.
- Keep moving to the right child.
- The rightmost node contains the maximum value.

6. Inorder Traversal
- Traverse **Left → Root → Right**.
- Gives BST elements in **sorted order**.

7. Preorder Traversal
- Traverse **Root → Left → Right**.

8. Postorder Traversal
- Traverse **Left → Right → Root**.

9. Count
- Counts the total number of nodes.
- Formula:
  Count = 1 + left subtree count + right subtree count

10. Height
- Finds the longest path from the root to a leaf.

- Formula:
  Height = 1 + max(left height, right height)

Important Points:

- root points to the first node.
- left stores the reference to the left child.
- right stores the reference to the right child.
- None represents an empty child.
- Inorder traversal gives **sorted elements**.
- Linked-list representation uses **dynamic memory**.

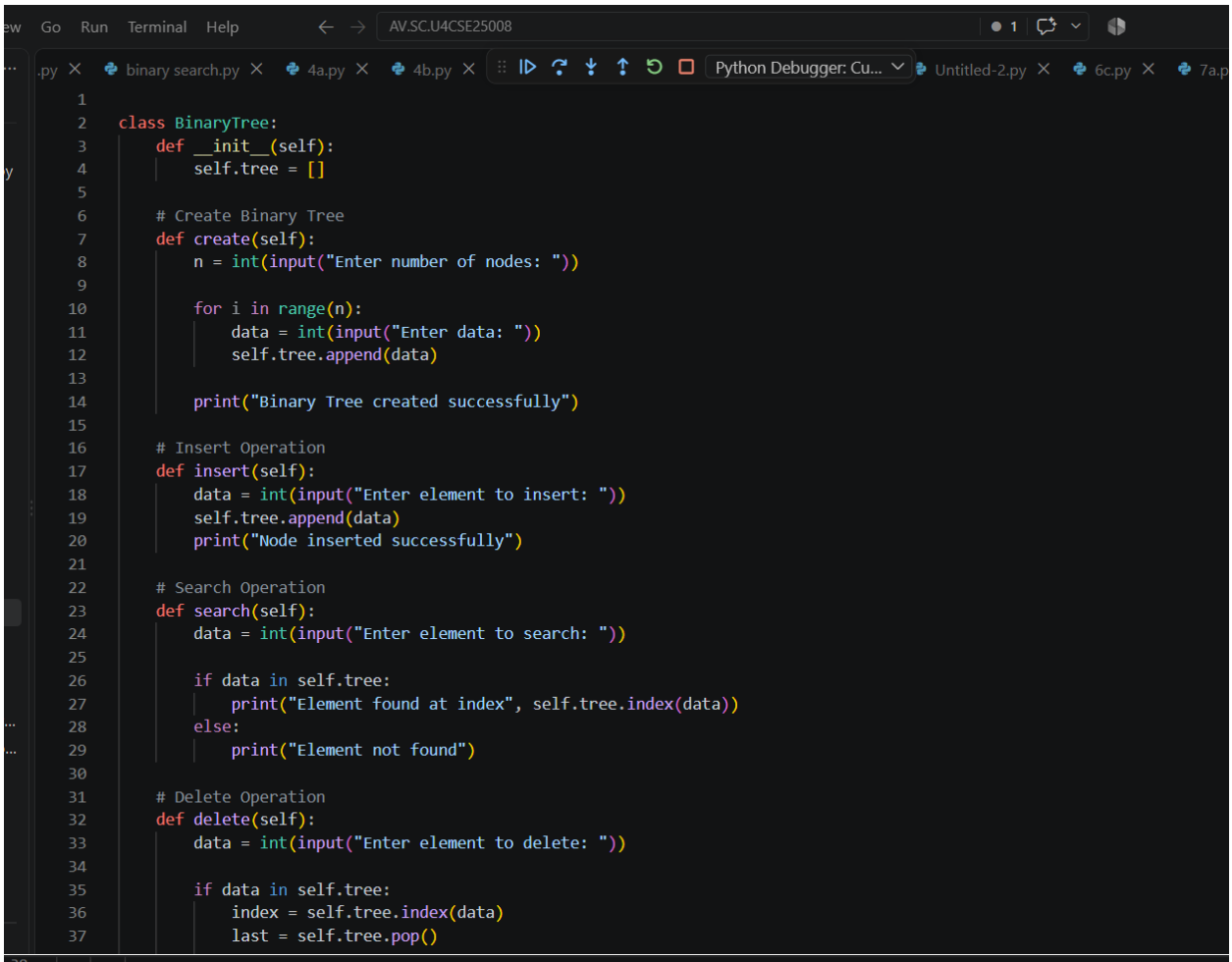
Error table:

S.NO	**Error Obtained**	**Rectification**
1.	Inserting the first node without setting root.	If self.root is None, set self.root = new and return.
2.	Searching for a value that does not exist.	Continue while temp is not None; if temp becomes None, return False.
3.	Incorrect movement while searching.	If data < temp.data, move to temp.left; otherwise move to temp.right.
4.	Deleting a node with two children incorrectly.	Find the minimum value in the right subtree, replace the node data, and delete the successor.

LAB 8

Aim : write a python program to implement Binary tree with its operations using array and linked list.

Program: using array




```python
class BinaryTree:
    def __init__(self):
        self.tree = []

    # Create Binary Tree
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            self.tree.append(data)

        print("Binary Tree created successfully")

    # Insert Operation
    def insert(self):
        data = int(input("Enter element to insert: "))
        self.tree.append(data)
        print("Node inserted successfully")

    # Search Operation
    def search(self):
        data = int(input("Enter element to search: "))

        if data in self.tree:
            print("Element found at index", self.tree.index(data))
        else:
            print("Element not found")

    # Delete Operation
    def delete(self):
        data = int(input("Enter element to delete: "))

        if data in self.tree:
            index = self.tree.index(data)
            last = self.tree.pop()

            if index < len(self.tree):
                self.tree[index] = last

            print("Node deleted successfully")
        else:
            print("Element not found")

    # Inorder Traversal
    def inorder(self, i=0):
        if i < len(self.tree):
            self.inorder(2 * i + 1)
            print(self.tree[i], end=" ")
            self.inorder(2 * i + 2)

    # Preorder Traversal
    def preorder(self, i=0):
        if i < len(self.tree):
            print(self.tree[i], end=" ")
            self.preorder(2 * i + 1)
            self.preorder(2 * i + 2)

    # Postorder Traversal
    def postorder(self, i=0):
        if i < len(self.tree):
            self.postorder(2 * i + 1)
            self.postorder(2 * i + 2)
            print(self.tree[i], end=" ")

    # Display Operation
    def display(self):
        if len(self.tree) == 0:
            print("Tree is empty")
        else:
            print("Binary Tree Array:", self.tree)
```

```python
# Main Program
bt = BinaryTree()

while True:
    print("\n----- BINARY TREE USING ARRAY -----")
    print("1. Create Binary Tree")
    print("2. Insert")
    print("3. Search")
    print("4. Delete")
    print("5. Inorder Traversal")
    print("6. Preorder Traversal")
    print("7. Postorder Traversal")
    print("8. Display")
    print("9. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        bt.create()
    elif choice == 2:
        bt.insert()
    elif choice == 3:
        bt.search()
    elif choice == 4:
        bt.delete()
    elif choice == 5:
        print("Inorder:", end=" ")
        bt.inorder()
        print()
    elif choice == 6:
        print("Preorder:", end=" ")
        bt.preorder()
        print()
    elif choice == 7:
        print("Postorder:", end=" ")
        bt.postorder()
        print()
```

Output:

```
Problems  Output  Debug Console  Terminal  Ports

2. Insert
3. Search
4. Delete
5. Inorder Traversal
6. Preorder Traversal
7. Postorder Traversal
8. Display
9. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 1
Enter data: 4
Enter data: 5
Binary Tree created successfully

----- BINARY TREE USING ARRAY -----
1. Create Binary Tree
2. Insert
3. Search
4. Delete
5. Inorder Traversal
6. Preorder Traversal
7. Postorder Traversal
8. Display
9. Exit
Enter your choice:
```

Explanation:

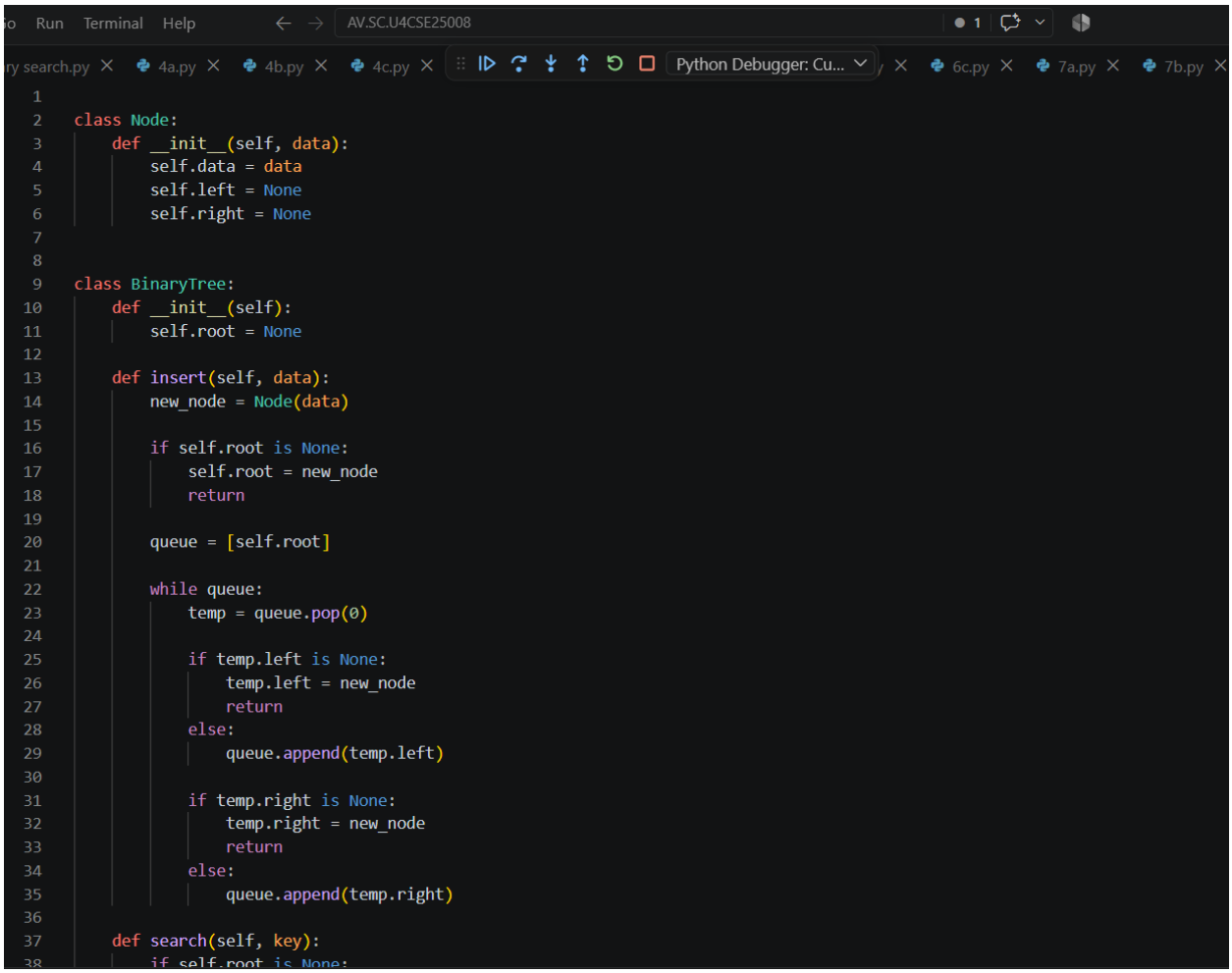
- The binarytree class is used to implement a binary tree using an array. The tree list is initialized with None values according to the given size.

- The insert_root() function stores the root node at index 0 because the root is always the first element in the array representation.
- The set_left() function is used to insert the left child of a node. For a node at index i, its left child is stored at index **2*i + 1**.
- The set_right() function is used to insert the right child of a node. For a node at index i, its right child is stored at index **2*i + 2**.
- The if child < self.size condition checks whether the calculated child index is within the available array size. If it exceeds the size, an index-out-of-range message is displayed.
- The preorder() function visits the current node first, followed by its left subtree and right subtree. The child indexes are calculated using 2*index + 1 and 2*index + 2.
- The inorder() function first visits the left subtree, then the current node and finally the right subtree.
- The postorder() function first visits the left subtree, then the right subtree and finally the current node.
- The levelorder() function visits the elements from the beginning to the end of the array. Since the array stores the binary tree level by level, this directly gives the level-order traversal.
- The display() function uses a loop to display every index of the array along with the value stored at that index.
- The condition self.tree[index] is None is used during traversal to stop when an empty position is reached.

Error Table:

S.No	Common Error	Correction
1	Not setting left and right references to None while creating a node.	Initialize both left and right as None in the Node constructor.
2	Calling traversal functions without passing the current node correctly.	Pass the current node during recursive calls.
3	Forgetting the base condition when the current node is None.	Use a condition such as if root is None: return.
4	Using the wrong traversal order.	Preorder: Root-Left-Right, Inorder: Left-Root-Right, Postorder: Left-Right-Root.

Program: using linked list




```python

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinaryTree:
    def __init__(self):
        self.root = None

    def insert(self, data):
        new_node = Node(data)

        if self.root is None:
            self.root = new_node
            return

        queue = [self.root]

        while queue:
            temp = queue.pop(0)

            if temp.left is None:
                temp.left = new_node
                return
            else:
                queue.append(temp.left)

            if temp.right is None:
                temp.right = new_node
                return
            else:
                queue.append(temp.right)

    def search(self, key):
        if self.root is None:
            return False

        queue = [self.root]

        while queue:
            temp = queue.pop(0)

            if temp.data == key:
                return True

            if temp.left:
                queue.append(temp.left)

            if temp.right:
                queue.append(temp.right)

        return False

    def delete(self, key):
        if self.root is None:
            print("Tree is empty")
            return

        queue = [(self.root, None)]
        target = None
        deepest = None
        parent = None

        while queue:
            temp, par = queue.pop(0)
            deepest = temp
            parent = par

            if temp.data == key and target is None:
                target = temp
```

```
Click to add a breakpoint
                temp.left:
                    queue.append((temp.left, temp))

                if temp.right:
                    queue.append((temp.right, temp))

        if target is None:
            print("Value not found")
            return

        target.data = deepest.data

        if parent is None:
            self.root = None
        elif parent.right == deepest:
            parent.right = None
        else:
            parent.left = None

        print("Node deleted")

    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)

    def preorder(self, node):
        if node:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    def postorder(self, node):
        if node:
            self.postorder(node.left)
```

```
            self.postorder(node.right)
            print(node.data, end=" ")

    def display(self):
        if self.root is None:
            print("Tree is empty")
            return

        queue = [self.root]

        while queue:
            temp = queue.pop(0)
            print(temp.data, end=" ")

            if temp.left:
                queue.append(temp.left)

            if temp.right:
                queue.append(temp.right)

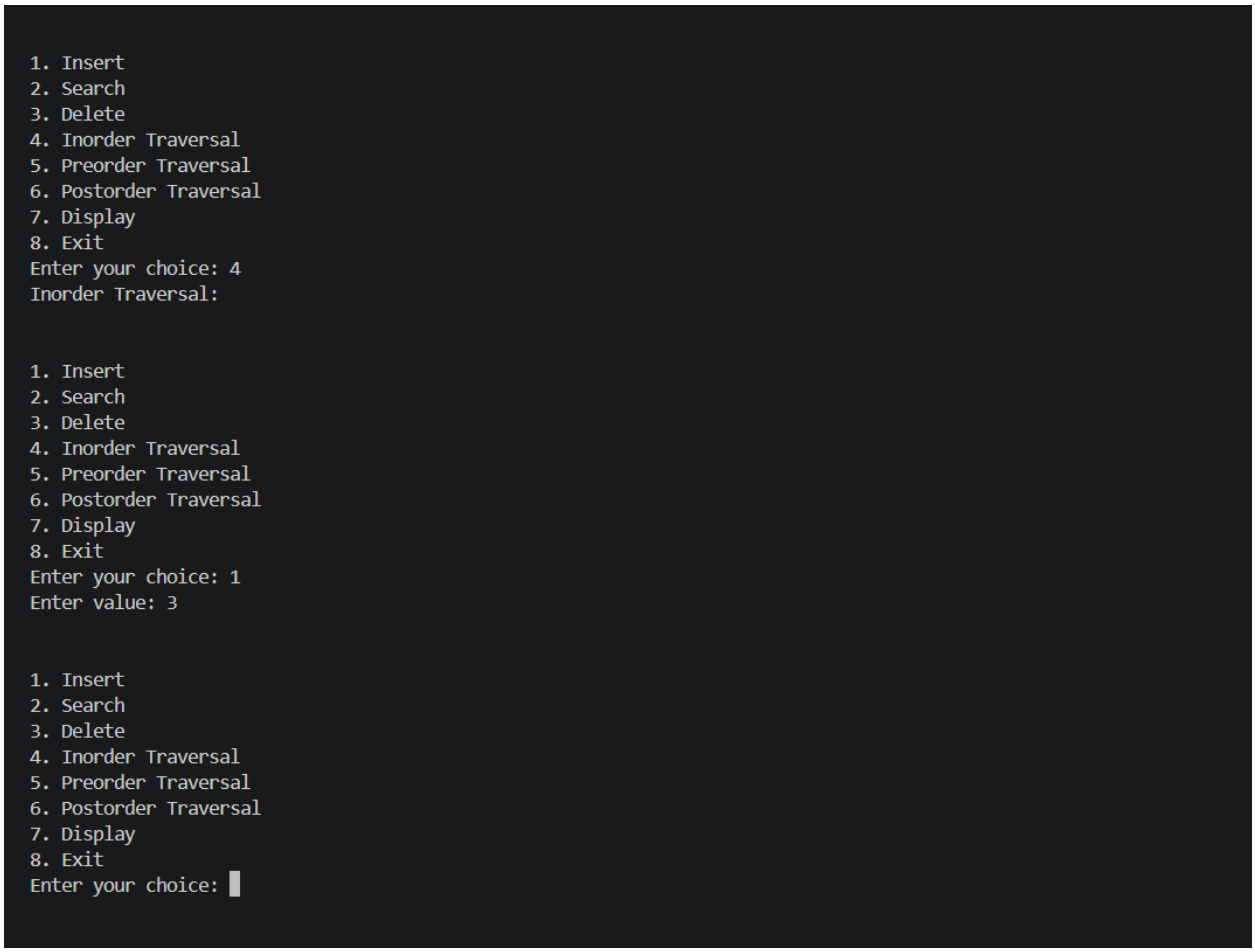

# Main program
bt = BinaryTree()

while True:
    print("\n\n1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Display")
    print("8. Exit")

    choice = int(input("Enter your choice: "))
```

```
146
147     if choice == 1:
148         value = int(input("Enter value: "))
149         bt.insert(value)
150
151     elif choice == 2:
152         value = int(input("Enter value to search: "))
153         if bt.search(value):
154             print("Value found")
155         else:
156             print("Value not found")
157
158     elif choice == 3:
159         value = int(input("Enter value to delete: "))
160         bt.delete(value)
161
162     elif choice == 4:
163         print("Inorder Traversal:")
164         bt.inorder(bt.root)
165
166     elif choice == 5:
167         print("Preorder Traversal:")
168         bt.preorder(bt.root)
169
170     elif choice == 6:
171         print("Postorder Traversal:")
172         bt.postorder(bt.root)
173
174     elif choice == 7:
175         print("Level Order:")
176         bt.display()
177
178     elif choice == 8:
179         print("Exiting...")
180         break
181
182     else:
183         print("Invalid choice")
```

Output:



Explanation:

- The Node class is used to create each node of the binary tree. Each node contains three parts: data, left, and right. The data stores the value, while left and right store references to the left and right child nodes.
- The __init__() method initializes the node with the given data. Initially, both left and right are set to None because the node does not have any children.
- The root represents the first node of the binary tree. New nodes are connected to the root using the left and right references.
- The preorder() function performs preorder traversal. It first visits the root node, then recursively visits the left subtree and finally the right subtree. The order followed is **Root → Left → Right**.
- The inorder() function performs inorder traversal. It first visits the left subtree, then the root node and finally the right subtree. The order followed is **Left → Root → Right**.
- The postorder() function performs postorder traversal. It first visits the left subtree, then the right subtree and finally the root node. The order followed is **Left → Right → Root**.
- The condition if root: checks whether the current node exists. If the node is None, the recursive function stops for that particular branch.

Error Table:

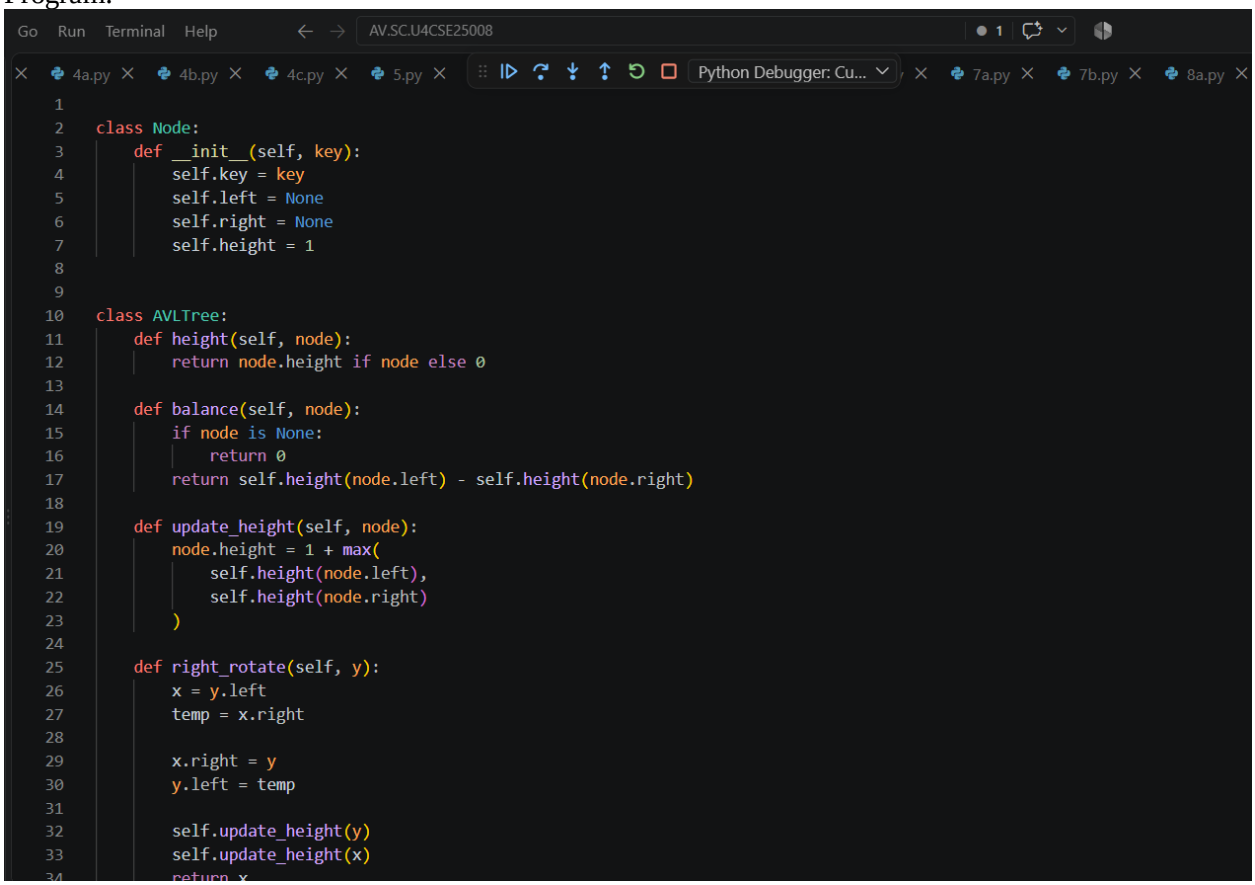
S.No	Common Error	Correction

1	Not setting left and right references to None while creating a node.	Initialize both left and right as None in the Node constructor.
2	Calling traversal functions without passing the current node correctly.	Pass the current node during recursive calls.
3	Forgetting the base condition when the current node is None.	Use a condition such as if root is None: return.
4	Using the wrong traversal order.	Preorder: Root-Left-Right, Inorder: Left-Root-Right, Postorder: Left-Right-Root.

Lab 9

Aim : write a python code for AVL tree using array and linked list with all its operations.

Program:



```
Click to add a breakpoint
    def left_rotate(self, x):
        y = x.right
        temp = y.left

        y.left = x
        x.right = temp

        self.update_height(x)
        self.update_height(y)
        return y

    def insert_node(self, node, key):
        if node is None:
            return Node(key)

        if key < node.key:
            node.left = self.insert_node(node.left, key)
        elif key > node.key:
            node.right = self.insert_node(node.right, key)
        else:
            return node

        self.update_height(node)
        balance = self.balance(node)

        # LL Case
        if balance > 1 and key < node.left.key:
            return self.right_rotate(node)

        # RR Case
        if balance < -1 and key > node.right.key:
            return self.left_rotate(node)

        # LR Case
        if balance > 1 and key > node.left.key:
            node.left = self.left_rotate(node.left)
            return self.right_rotate(node)

        # RL Case
        if balance < -1 and key < node.right.key:
            node.right = self.right_rotate(node.right)
            return self.left_rotate(node)

        return node

    def min_value_node(self, node):
        current = node
        while current.left:
            current = current.left
        return current

    def delete_node(self, node, key):
        if node is None:
            return None

        if key < node.key:
            node.left = self.delete_node(node.left, key)
        elif key > node.key:
            node.right = self.delete_node(node.right, key)
        else:
            if node.left is None:
                return node.right
            elif node.right is None:
                return node.left

            temp = self.min_value_node(node.right)
            node.key = temp.key
            node.right = self.delete_node(node.right, temp.key)

        self.update_height(node)
```

```
        balance = self.balance(node)

        # LL Case
        if balance > 1 and self.balance(node.left) >= 0:
            return self.right_rotate(node)

        # LR Case
        if balance > 1 and self.balance(node.left) < 0:
            node.left = self.left_rotate(node.left)
            return self.right_rotate(node)

        # RR Case
        if balance < -1 and self.balance(node.right) <= 0:
            return self.left_rotate(node)

        # RL Case
        if balance < -1 and self.balance(node.right) > 0:
            node.right = self.right_rotate(node.right)
            return self.left_rotate(node)

        return node

    def search(self, node, key):
        if node is None or node.key == key:
            return node

        if key < node.key:
            return self.search(node.left, key)

        return self.search(node.right, key)

    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.key, end=" ")
            self.inorder(node.right)

    def preorder(self, node):
        if node:
            print(node.key, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    def postorder(self, node):
        if node:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.key, end=" ")

    def display(self, node):
        if node:
            print(node.key, end=" ")
            self.display(node.left)
            self.display(node.right)


tree = AVLTree()
root = None

while True:
    print("\n1.Insert  2.Delete  3.Search")
    print("4.Inorder  5.Preorder  6.Postorder")
    print("7.Display  8.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        key = int(input("Enter value: "))
        root = tree.insert_node(root, key)

```

```
    elif choice == 2:
        key = int(input("Enter value to delete: "))
        if tree.search(root, key):
            root = tree.delete_node(root, key)
            print("Node deleted")
        else:
            print("Value not found")

    elif choice == 3:
        key = int(input("Enter value to search: "))
        if tree.search(root, key):
            print("Value found")
        else:
            print("Value not found")

    elif choice == 4:
        tree.inorder(root)
        print()

    elif choice == 5:
        tree.preorder(root)
        print()

    elif choice == 6:
        tree.postorder(root)
        print()

    elif choice == 7:
        tree.display(root)
        print()

    elif choice == 8:
        print("Exiting...")
        break

    else:
        print("Invalid choice")
```

AVL Using II.py - C:/Users/deepthika/AppData/Local/Programs/Python/Python313/AVL Using II.py (3.13.7)

File Edit Format Run Options Window Help

```
                root.data = temp.data

                root.right = self.delete(
                    root.right,
                    temp.data
                )

        self.update_height(root)

        balance = self.get_balance(root)

        # LL Case
        if balance > 1 and self.get_balance(root.left) >= 0:
            return self.right_rotate(root)

        # LR Case
        if balance > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        # RR Case
        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)

        # RL Case
        if balance < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root

    # Search
    def search(self, root, data):

        if root is None:
            return False

        if root.data == data:
            return True

        if data < root.data:
            return self.search(root.left, data)

        return self.search(root.right, data)

    # Inorder
```

AVL Using II.py - C:/Users/deepthika/AppData/Local/Programs/Python/Python313/AVL Using II.py (3.13.7)

File Edit Format Run Options Window Help

```
    # Inorder
    def inorder(self, root):

        if root is not None:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    # Preorder
    def preorder(self, root):

        if root is not None:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    # Postorder
    def postorder(self, root):

        if root is not None:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")

    # Display tree
    def display(self, root, level=0):

        if root is not None:
            self.display(root.right, level + 1)

            print("    " * level + str(root.data))

            self.display(root.left, level + 1)


# Main program

avl = AVLTree()
root = None

while True:

    print("\n--- AVL TREE USING LINKED LIST ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
```

AVL Using II.py - C:/Users/deepthika/AppData/Local/Programs/Python/Python313/AVL Using II.py (3.13.7)

File Edit Format Run Options Window Help

```
avl = AVLTree()
root = None

while True:

    print("\n--- AVL TREE USING LINKED LIST ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder")
    print("5. Preorder")
    print("6. Postorder")
    print("7. Display")
    print("8. Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        data = int(input("Enter value: "))
        root = avl.insert(root, data)
    elif choice == 2:
        data = int(input("Enter value: "))
        root = avl.delete(root, data)
    elif choice == 3:
        data = int(input("Enter value: "))
        if avl.search(root, data):
            print("Element found")
        else:
            print("Element not found")
    elif choice == 4:
        print("Inorder:", end=" ")
        avl.inorder(root)
        print()
    elif choice == 5:
        print("Preorder:", end=" ")
        avl.preorder(root)
        print()
    elif choice == 6:
        print("Postorder:", end=" ")
        avl.postorder(root)
        print()
    elif choice == 7:
        print("\nAVL Tree:")
        avl.display(root)
    elif choice == 8:
        break
    else:
        print("Invalid choice")
```

Output:

```
1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder
7.Display  8.Exit
Enter choice: 3
Enter value to search: 4
Value not found

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder
7.Display  8.Exit
Enter choice: 5


1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder
7.Display  8.Exit
Enter choice:
```

Explanation:

- An **AVL tree** is a self-balancing Binary Search Tree (BST).
- In an AVL tree, the difference between the height of the left and right subtrees is called the **Balance Factor**.
- The Balance Factor of every node must be **-1, 0, or +1**.

- In the array implementation, the tree nodes are stored inside a fixed-size array.
- For a node at index i:
    - Left child is stored at 2*i + 1.
    - Right child is stored at 2*i + 2.
- None is used to represent an empty position in the array.

Main Functions

- **__init__()**
    - Creates the array for storing the tree.
    - The size of the array is given by the user/program.
    - Initially, all positions contain None.
- **height()**
    - Calculates the height of a node.
    - It recursively checks the left and right subtrees.
    - The greater height is selected and 1 is added.
- **balance_factor()**
    - Calculates the balance factor of a node.
    - It is calculated as:
        - Height of Left Subtree - Height of Right Subtree
    - A value greater than 1 indicates that the tree is left-heavy.
    - A value less than -1 indicates that the tree is right-heavy.
- **insert()**
    - Adds a new element to the AVL tree.
    - The element is placed according to Binary Search Tree rules.
    - Smaller values are placed in the left subtree.
    - Larger values are placed in the right subtree.
    - After insertion, the tree is checked and rotations are performed if required.
- **search()**
    - Searches for a particular element.
    - If the element is found, it returns True.
    - If the element does not exist, it returns False.
- **delete()**
    - Removes an element from the tree.

    - It handles nodes with no child, one child, or two children.
    - After deletion, the tree is rebalanced.
- **find_min()**
    - Finds the smallest element in a subtree.
    - It repeatedly moves to the left child.
    - It is mainly used when deleting a node having two children.
- **rotate_left()**
    - Performs a left rotation when the tree becomes right-heavy.
    - It helps restore the AVL property.
- **rotate_right()**
    - Performs a right rotation when the tree becomes left-heavy.
    - It helps restore the AVL property.
- **balance_tree()**
    - Checks the balance factor of the nodes.
    - If the tree becomes unbalanced, suitable rotations are performed.
- **inorder()**
    - Traverses the tree in:
        - Left → Root → Right
    - It displays the elements in sorted order.
- **preorder()**
    - Traverses the tree in:
        - Root → Left → Right
- **postorder()**
    - Traverses the tree in:
        - Left → Right → Root
- **display()**
    - Displays the elements along with their array indexes.
    - It helps to understand how the tree is stored in the array.

Main Program

- The main program provides a **menu-driven interface**.
- The user can choose:
    - Insert

- Delete
- Search
- Inorder
- Preorder
- Postorder
- Display
- Exit

- This allows all major AVL tree operations to be performed interactively.

---

2. AVL Tree Using Linked List – Explanation

- In the linked-list implementation, every tree element is represented using a **Node**.
- Each node contains:
  - Data
  - Left child
  - Right child
  - Height
- The left and right pointers connect the nodes and form the tree.
- Unlike the array implementation, memory is allocated dynamically as new nodes are created.

Node Class

- The Node class represents one node of the AVL tree.
- data stores the value.
- left stores the reference to the left child.
- right stores the reference to the right child.
- height stores the height of the node.
- Initially, a new node has height 1.

Main Functions

- **height()**
  - Returns the height of a node.
  - If the node is None, its height is considered 0.
- **get_balance()**
  - Calculates the balance factor.
  - Formula:
    - Height(Left) - Height(Right)

    - It is used to determine whether rotations are required.
- **update_height()**
    - Updates the height of a node after insertion, deletion, or rotation.
    - It takes the maximum height of the left and right subtrees and adds 1.
- **right_rotate()**
    - Performs a right rotation.
    - It is mainly used for the **LL imbalance**.
    - The left child becomes the new root of that subtree.
- **left_rotate()**
    - Performs a left rotation.
    - It is mainly used for the **RR imbalance**.
    - The right child becomes the new root of that subtree.
- **insert()**
    - Inserts a new value according to BST rules.
    - Smaller values go to the left.
    - Larger values go to the right.
    - The height of each affected node is updated.
    - The balance factor is checked.
    - If the tree becomes unbalanced, the appropriate rotation is performed.

Four Cases During Insertion

- **LL Case**
    - New node is inserted into the left subtree of the left child.
    - A **right rotation** is performed.
- **RR Case**
    - New node is inserted into the right subtree of the right child.
    - A **left rotation** is performed.
- **LR Case**
    - New node is inserted into the right subtree of the left child.
    - First, a **left rotation** is performed on the left child.
    - Then, a **right rotation** is performed.
- **RL Case**
    - New node is inserted into the left subtree of the right child.

    - First, a **right rotation** is performed on the right child.
    - Then, a **left rotation** is performed.
- **min_value_node()**
    - Finds the node with the minimum value.
    - It keeps moving to the left until the left child is None.
    - It is used during deletion of a node with two children.
- **delete()**
    - Deletes a node from the AVL tree.
    - Three situations are handled:
        - Node has no child.
        - Node has one child.
        - Node has two children.
    - For two children, the inorder successor is used.
    - After deletion, heights are updated.
    - The balance factor is checked and rotations are performed if required.
- **search()**
    - Searches for a given value.
    - If the value matches the current node, it returns True.
    - If the value is smaller, it searches the left subtree.
    - If the value is larger, it searches the right subtree.
- **inorder()**
    - Traverses:
        - Left → Root → Right
    - Produces the elements in sorted order.
- **preorder()**
    - Traverses:
        - Root → Left → Right
- **postorder()**
    - Traverses:
        - Left → Right → Root
- **display()**

- Displays the tree structure vertically.
- It helps visualize the relationship between parent and child nodes.

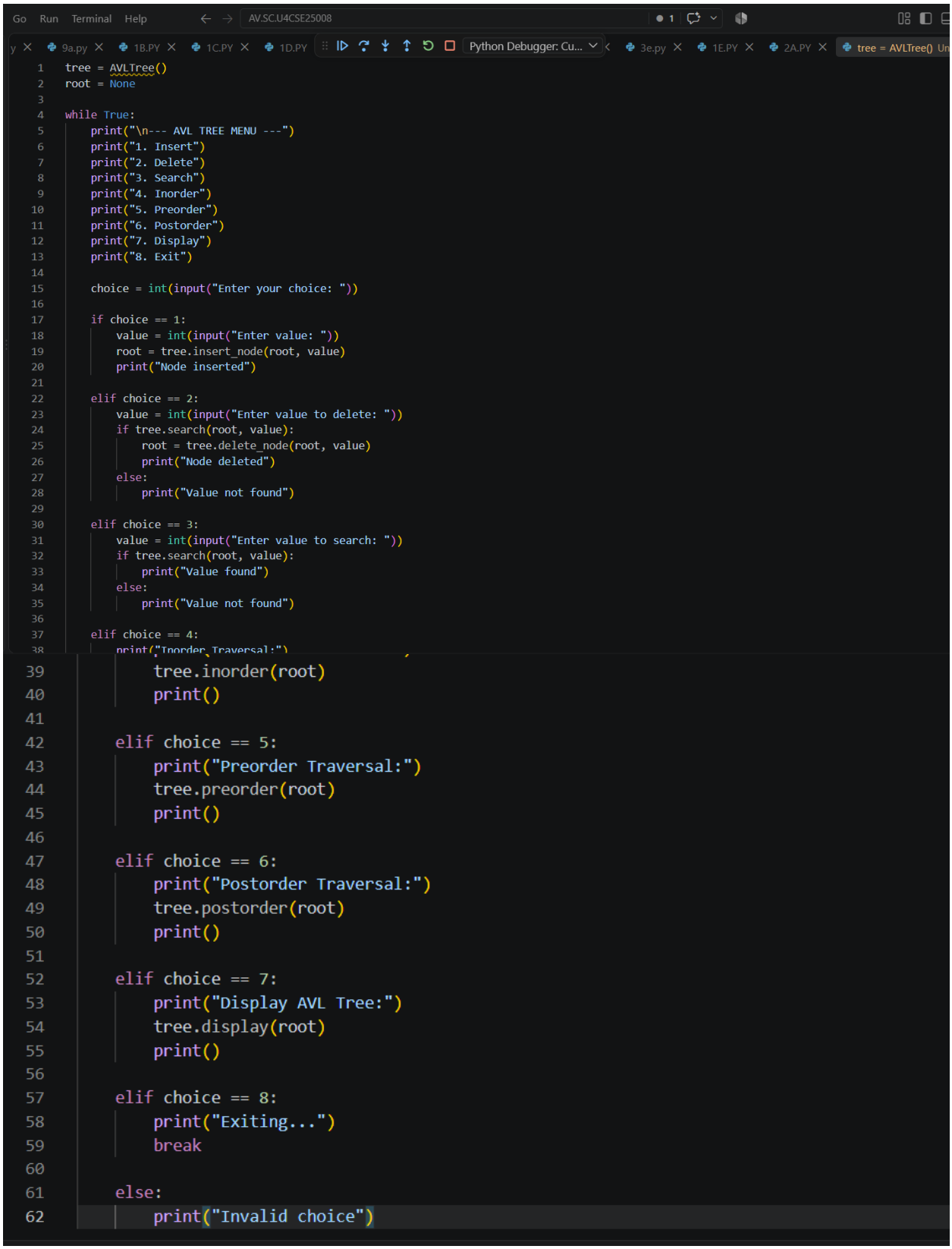
Main Program

- The main program creates an AVL tree object.
- root = None represents an initially empty tree.
- A menu is provided to perform different operations.
- The user can perform:
  - Insert
  - Delete
  - Search
  - Inorder
  - Preorder
  - Postorder
  - Display
  - Exit

Error Table:

S.No.	Common Error	Reason / Correction
1	AttributeError: NoneType	The code tries to access a property of an empty node. Check whether the node is None before accessing its data or children.
2	Incorrect rotation	The left/right child references are changed incorrectly. Verify the temporary node and child links during rotation.
3	Wrong height or balance factor	The height is not updated after insertion, deletion, or rotation. Call the height update function at the correct place.
4	Element not found during deletion/search	The value does not exist in the tree or the search path is incorrect. Follow BST comparison rules correctly.

Program:



```
tree = AVLTree()
root = None

while True:
    print("\n--- AVL TREE MENU ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder")
    print("5. Preorder")
    print("6. Postorder")
    print("7. Display")
    print("8. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        root = tree.insert_node(root, value)
        print("Node inserted")

    elif choice == 2:
        value = int(input("Enter value to delete: "))
        if tree.search(root, value):
            root = tree.delete_node(root, value)
            print("Node deleted")
        else:
            print("Value not found")

    elif choice == 3:
        value = int(input("Enter value to search: "))
        if tree.search(root, value):
            print("Value found")
        else:
            print("Value not found")

    elif choice == 4:
        print("Inorder Traversal:")
        tree.inorder(root)
        print()

    elif choice == 5:
        print("Preorder Traversal:")
        tree.preorder(root)
        print()

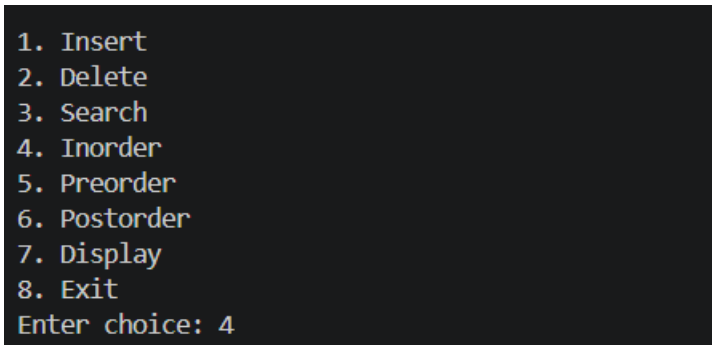
    elif choice == 6:
        print("Postorder Traversal:")
        tree.postorder(root)
        print()

    elif choice == 7:
        print("Display AVL Tree:")
        tree.display(root)
        print()

    elif choice == 8:
        print("Exiting...")
        break

    else:
        print("Invalid choice")
```

Output:



Explanation:

- An **AVL tree** is a self-balancing Binary Search Tree (BST).
- In an AVL tree, the difference between the height of the left and right subtrees is called the **Balance Factor**.
- The Balance Factor of every node must be **-1, 0, or +1**.
- In the array implementation, the tree nodes are stored inside a fixed-size array.
- For a node at index i:
    - Left child is stored at 2*i + 1.
    - Right child is stored at 2*i + 2.
- None is used to represent an empty position in the array.

Main Functions

- **__init__()**
    - Creates the array for storing the tree.
    - The size of the array is given by the user/program.
    - Initially, all positions contain None.
- **height()**
    - Calculates the height of a node.
    - It recursively checks the left and right subtrees.
    - The greater height is selected and 1 is added.
- **balance_factor()**
    - Calculates the balance factor of a node.
    - It is calculated as:
        - Height of Left Subtree - Height of Right Subtree
    - A value greater than 1 indicates that the tree is left-heavy.
    - A value less than -1 indicates that the tree is right-heavy.
- **insert()**
    - Adds a new element to the AVL tree.

    - The element is placed according to Binary Search Tree rules.
    - Smaller values are placed in the left subtree.
    - Larger values are placed in the right subtree.
    - After insertion, the tree is checked and rotations are performed if required.
- **search()**
    - Searches for a particular element.
    - If the element is found, it returns True.
    - If the element does not exist, it returns False.
- **delete()**
    - Removes an element from the tree.
    - It handles nodes with no child, one child, or two children.
    - After deletion, the tree is rebalanced.
- **find_min()**
    - Finds the smallest element in a subtree.
    - It repeatedly moves to the left child.
    - It is mainly used when deleting a node having two children.
- **rotate_left()**
    - Performs a left rotation when the tree becomes right-heavy.
    - It helps restore the AVL property.
- **rotate_right()**
    - Performs a right rotation when the tree becomes left-heavy.
    - It helps restore the AVL property.
- **balance_tree()**
    - Checks the balance factor of the nodes.
    - If the tree becomes unbalanced, suitable rotations are performed.
- **inorder()**
    - Traverses the tree in:
        - Left → Root → Right
    - It displays the elements in sorted order.
- **preorder()**
    - Traverses the tree in:

        - Root → Left → Right
- **postorder()**
    - Traverses the tree in:
        - Left → Right → Root
- **display()**
    - Displays the elements along with their array indexes.
    - It helps to understand how the tree is stored in the array.

Main Program

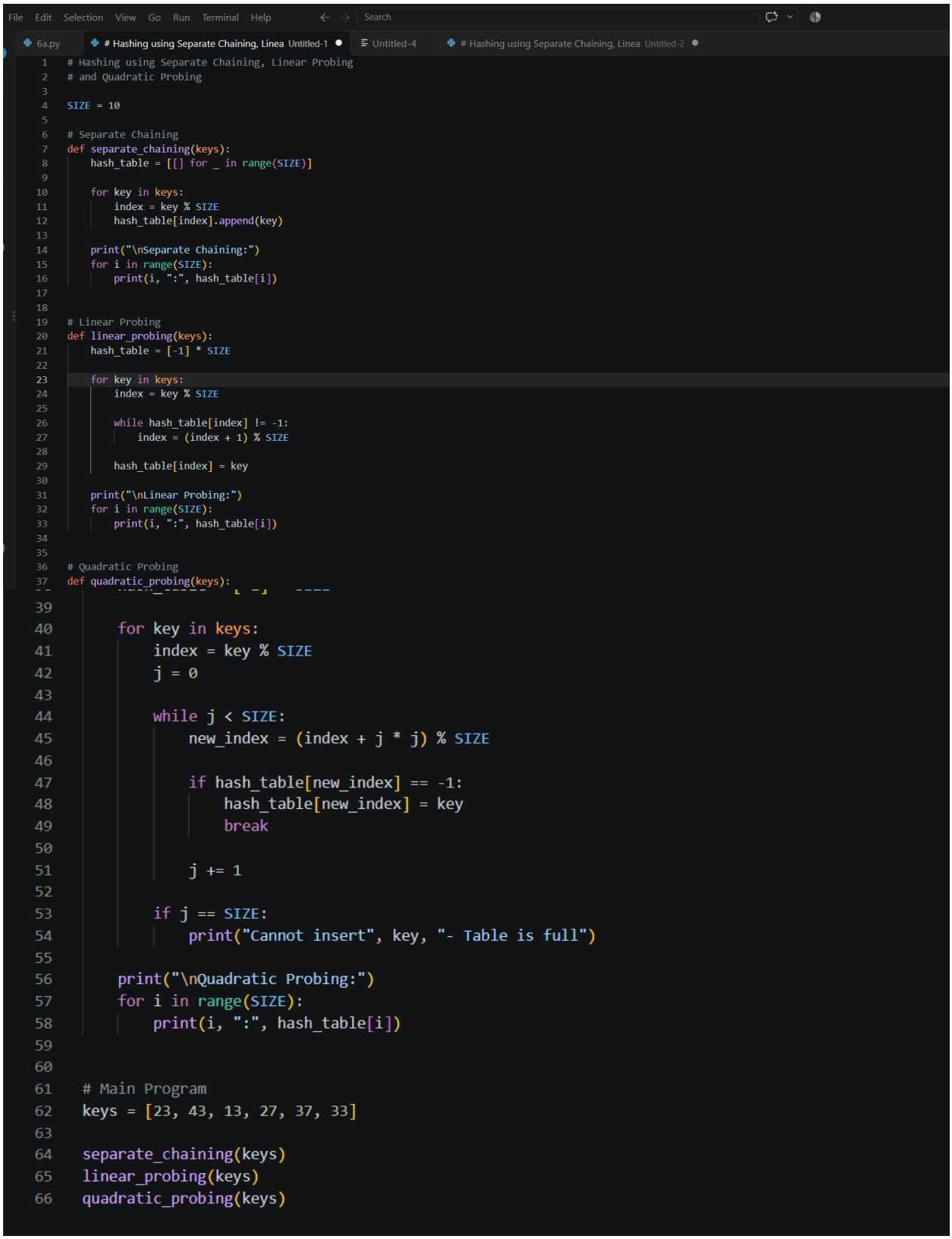
- The main program provides a **menu-driven interface**.
- The user can choose:
    - Insert
    - Delete
    - Search
    - Inorder
    - Preorder
    - Postorder
    - Display
    - Exit
- This allows all major AVL tree operations to be performed interactively.

Error table:

S.No.	Common Error	Reason / Correction
1	Index out of range	The calculated child index exceeds the array size. Use a sufficiently large array and check the index before accessing it.
2	Tree becomes full	The fixed-size array has no empty position available. Increase the array size.
3	Incorrect child index	Using the wrong formula for child positions causes incorrect tree structure. Use left = 2*i+1 and right = 2*i+2.
4	Incorrect balancing after insertion/deletion	The balance factor or rotation is not handled correctly. Check the balance factor and perform the required rotation.

WEEK-10

Aim: Write a Python program to implement Hashing using Separate Chaining, Linear Probing, and Quadratic Probing. Program:




```python
# Hashing using Separate Chaining, Linear Probing
# and Quadratic Probing

SIZE = 10

# Separate Chaining
def separate_chaining(keys):
    hash_table = [[] for _ in range(SIZE)]

    for key in keys:
        index = key % SIZE
        hash_table[index].append(key)

    print("\nSeparate Chaining:")
    for i in range(SIZE):
        print(i, ":", hash_table[i])


# Linear Probing
def linear_probing(keys):
    hash_table = [-1] * SIZE

    for key in keys:
        index = key % SIZE

        while hash_table[index] != -1:
            index = (index + 1) % SIZE

        hash_table[index] = key

    print("\nLinear Probing:")
    for i in range(SIZE):
        print(i, ":", hash_table[i])


# Quadratic Probing
def quadratic_probing(keys):

    for key in keys:
        index = key % SIZE
        j = 0

        while j < SIZE:
            new_index = (index + j * j) % SIZE

            if hash_table[new_index] == -1:
                hash_table[new_index] = key
                break

            j += 1

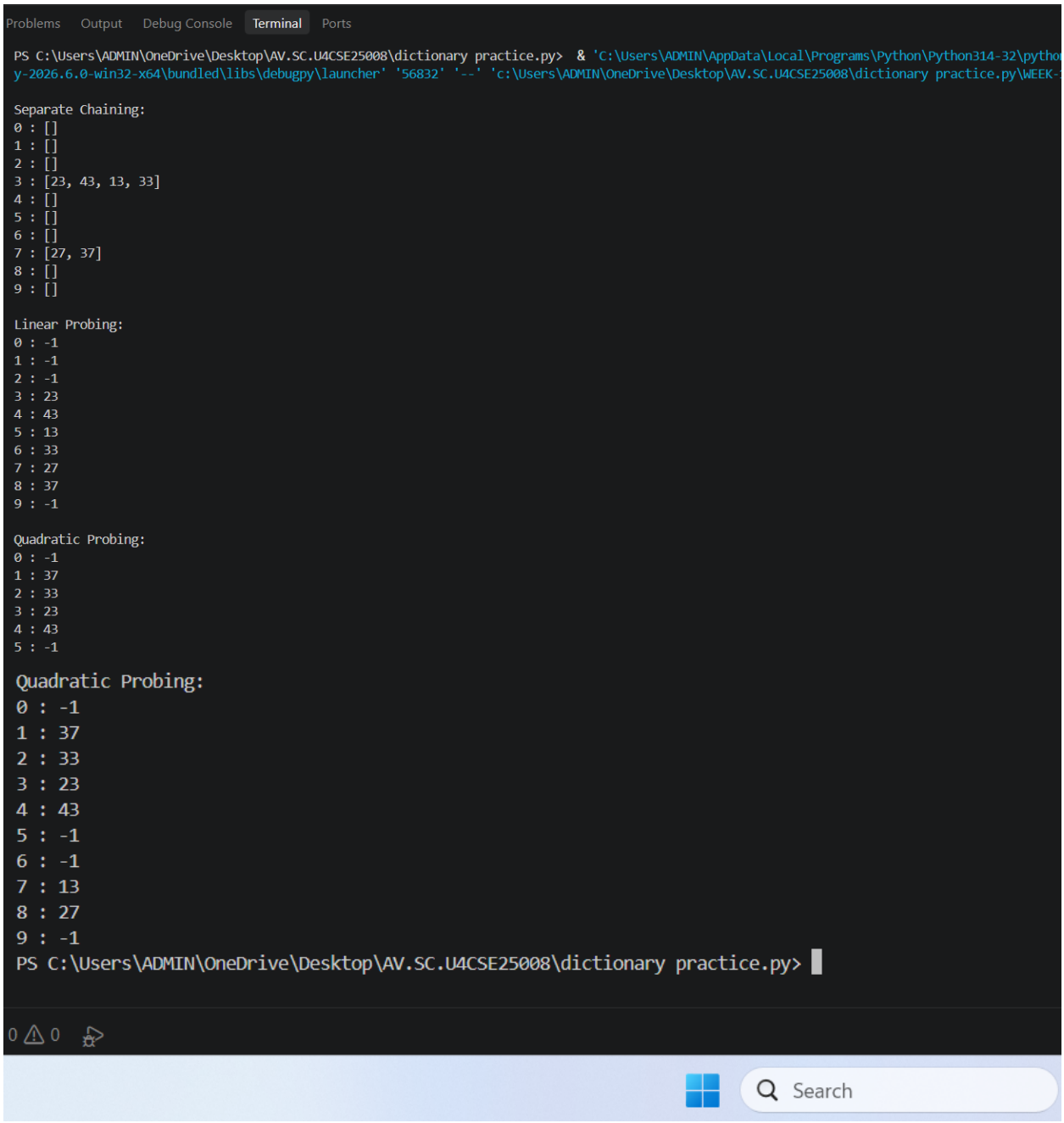
        if j == SIZE:
            print("Cannot insert", key, "- Table is full")

    print("\nQuadratic Probing:")
    for i in range(SIZE):
        print(i, ":", hash_table[i])


# Main Program
keys = [23, 43, 13, 27, 37, 33]

separate_chaining(keys)
linear_probing(keys)
quadratic_probing(keys)
```

OUTPUT:



Explanation:

- The program implements three hashing techniques: Separate Chaining, Linear Probing, and Quadratic Probing.
- The SIZE = 11 statement defines the size of the hash table as 11.
- The chaining list stores elements using separate chaining.
- The linear list stores elements using linear probing.
- The quadratic list stores elements using quadratic probing.
- The insert_chaining() function inserts an element into the corresponding index using the modulo operation.
- The search_chaining() function searches for an element in the corresponding chain.
- The display_chaining() function displays all the elements stored in the separate chaining hash table.
- The insert_linear() function inserts an element by checking consecutive positions when a collision occurs.
- The search_linear() function searches for an element using linear probing.

- The display_linear() function displays the elements in the linear probing hash table.
- The insert_quadratic() function resolves collisions by checking positions using the quadratic formula.
- The search_quadratic() function searches for an element using quadratic probing.
- The display_quadratic() function displays the elements in the quadratic probing hash table.
- The modulo operation key % SIZE calculates the initial hash index.
- The None value indicates an empty position in the linear and quadratic probing tables.
- The main menu allows the user to select a hashing technique or exit the program.
- The submenu allows the user to insert, search, display elements, or return to the main menu.
- The if-elif-else statements execute the selected operation.
- The program demonstrates how collisions are handled using three different hashing techniques.

Error table:

sno	Error obtained	Error rectification
1	Collision occurs when two elements have the same hash index.	Used separate chaining, linear probing, and quadratic probing to handle collisions.
2	Incorrect hash index calculation.	Used the modulo operation key % SIZE to calculate the hash index.